Machine Learning

The Complete Guide for GenAI Engineers — every classical algorithm in depth: what it is, how it works, when to use it

Inside: 14 chapters • ML overview for GenAI engineers • end-to-end pipeline (hands-on) • evaluation, persistence & ML as a Service (FastAPI) • Decision Trees (worked Gini/entropy example + from-scratch code) • Random Forest (variance math, OOB, importance) • SVM theory, soft margin, dual, kernels, C & γ • Linear/Ridge/Lasso, Logistic, KNN, Naive Bayes, clustering • scenario → algorithm guide • multi-model comparison & tuning (Grid, Random, Optuna, nested CV) • bias–variance & optimisation + practice assessment • feature engineering & PCA • bagging, AdaBoost, Gradient Boosting, XGBoost, LightGBM, CatBoost, stacking • improvement strategies • ~70 interview Q&A • cheat sheet

Prepared for Mansavi Aggarwal

Contents

1. Machine Learning Overview for GenAI Engineers **5**

1.1 What is Machine Learning? . . . 5

1.2 Why classical ML matters for GenAI engineers . . . 5

1.3 Types of learning . . . 6

1.4 Key vocabulary . . . 6

1.5 The 'No Free Lunch' theorem . . . 7

2. End-to-End ML Pipeline (Hands-on) **8**

2.1 Step 1 — Frame the problem . . . 8

2.2 Step 2 — Data collection & EDA . . . 8

2.3 Step 3 — Split before you transform . . . 9

2.4 Step 4 — Preprocess with Pipeline & ColumnTransformer . . . 9

2.5 Step 5 — Train, evaluate, iterate . . . 10

2.6 Steps 6–8 — Persist, serve, monitor . . . 10

3. Model Evaluation, Persistence & ML as a Service **11**

3.1 Validation strategies . . . 11

3.2 Classification metrics . . . 12

3.3 Regression metrics . . . 13

3.4 Model persistence . . . 13

3.5 ML as a Service with FastAPI . . . 14

3.6 Monitoring after deployment . . . 15

4. Decision Trees: Algorithm & Implementation **16**

4.1 Intuition . . . 16

4.2 Measuring impurity . . . 17

4.3 The CART algorithm (Classification And Regression Trees) . . . 17

4.4 Worked example — choosing the root split . . . 17

4.5 Regression trees . . . 18

4.6 Overfitting, stopping and pruning . . . 18

4.7 Implementation from scratch . . . 19

4.8 Decision trees in scikit-learn . . . 20

4.9 Strengths, weaknesses and when to use . . . 20

5. Random Forest: an Ensemble of Trees **21**

5.1 Intuition . . . 21

5.2 The algorithm . . . 21

5.3 Why averaging works (the math) . . . 21

5.4 Feature importance . . . 22
5.5 Hyper-parameters . . . 23
5.6 Random Forest from scratch (using our tree) . . . 23
5.7 Extra Trees and other variants . . . 23
5.8 When to use Random Forest . . . 24

6. Support Vector Machines: Theory & Kernels 25

6.1 Intuition: the widest street . . . 25
6.2 Hard-margin SVM (separable data) . . . 25
6.3 Soft margin, C and hinge loss . . . 26
6.4 The dual problem and why it matters . . . 26
6.5 The kernel trick . . . 26
6.6 Practical details . . . 27
6.7 When to use SVM . . . 28

7. Classical ML Models Overview & When to Use Them 29

7.1 Linear Regression . . . 29
7.2 Logistic Regression . . . 30
7.3 K-Nearest Neighbours (KNN) . . . 31
7.4 Naive Bayes . . . 31
7.5 Trees, Random Forest, SVM — recap . . . 32
7.6 Unsupervised algorithms . . . 32
7.7 Master comparison table . . . 33
7.8 Scenario → algorithm guide . . . 34

8. Hands-on: Multi-Model Comparison & Tuning 35

8.1 Protocol for a fair comparison . . . 35
8.2 Code: compare many models . . . 35
8.3 Hyper-parameter tuning methods . . . 36
8.4 Tuning cheat sheet per algorithm . . . 36
8.5 Nested cross-validation and final selection . . . 37

9. Bias–Variance, Overfitting & Practical Optimisation 38

9.1 The bias–variance decomposition . . . 38
9.2 Diagnosing your model . . . 39
9.3 Complexity knobs per algorithm . . . 39
9.4 Practical optimisation: how models are trained . . . 40
9.5 Practice assessment (milestone-style) . . . 41

10. Feature Engineering & Dimensionality Reduction 42

10.1 Numeric features . . . 42

10.2 Categorical features . . . 43
10.3 Date-time, text and other features . . . 43
10.4 Missing values, outliers and imbalance . . . 43
10.5 Feature selection . . . 43
10.6 Dimensionality reduction . . . 44

11. Ensemble Learning: Bagging & Boosting 46

11.1 Why combining models works . . . 46
11.2 Voting . . . 47
11.3 Bagging . . . 47
11.4 AdaBoost (Adaptive Boosting) . . . 47
11.5 Gradient Boosting . . . 48
11.6 XGBoost, LightGBM, CatBoost . . . 48
11.7 Stacking and blending . . . 49
11.8 Which ensemble when? . . . 49

12. Practical Model Improvement Strategies 50

12.1 The improvement loop . . . 50
12.2 Data-centric strategies . . . 50
12.3 Feature-centric strategies . . . 50
12.4 Model-centric strategies . . . 51
12.5 Decision-level strategies . . . 51
12.6 Error analysis & interpretability . . . 51
12.7 Reproducibility & experiment tracking . . . 51
12.8 Improvement checklist . . . 51

13. Interview Questions & Answers 52

14. Cheat Sheet 57

14.1 Formulas . . . 57
14.2 Algorithms at a glance . . . 58

CHAPTER 1

Machine Learning Overview for GenAI Engineers

What ML is • why GenAI engineers need classical ML • types of learning • key vocabulary • where classical ML sits inside GenAI systems • the ML algorithm landscape

1.1 What is Machine Learning?

Machine Learning (ML) is the practice of building systems that **learn patterns from data** instead of following hand-written rules. Traditional programming: *rules + data → answers*. Machine learning: *data + answers → rules (a model)*, which is then applied to new data.

Tom Mitchell's definition: a program learns from experience **E** with respect to task **T** and performance measure **P** if its performance on T, measured by P, improves with E. Example — spam filter: T = classify emails, E = labelled emails, P = F1 on unseen emails.

$$\hat{f} = \arg\min_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} L(y_i, f(x_i)) + \lambda\, \Omega(f)$$

almost every ML algorithm = choose a hypothesis space H, a loss L, a regulariser Ω, and an optimiser

1.2 Why classical ML matters for GenAI engineers

LLMs did not replace classical ML — they sit next to it. In real GenAI systems you will constantly use classical ML ideas and models:

Where	Classical ML used	Example
Routing & guardrails	fast classifiers (Logistic Regression, XGBoost) on embeddings or TF-IDF	route queries to the right agent; detect toxic/PII inputs cheaply before calling an LLM
Retrieval (RAG)	KNN / approximate nearest neighbours, BM25, re-rankers	vector search is k-nearest-neighbours in embedding space
Evaluation	precision/recall/F1, ROC, cross-validation, statistical tests	evaluating an LLM classifier, RAG retrieval recall@k
Embeddings + ML	LLM/sentence embeddings as features → Logistic Regression / SVM / clustering	intent classification with 500 labels at 1/100th the cost of an LLM call
Clustering & topic discovery	K-Means, HDBSCAN, PCA/UMAP	group user queries to find gaps in a knowledge base
Tabular prediction	gradient boosting (XGBoost, LightGBM)	LLMs are poor at numeric tabular prediction; GBMs are the state of the art
Cost/latency control	small models instead of LLM calls	pre-filter 95% of easy cases, send 5% to the LLM
Concepts	loss, overfitting, bias–variance, regularisation, data leakage	fine-tuning LLMs uses exactly the same ideas

Interview tip: Interview line: *"I use LLMs where language understanding or generation is needed, and classical ML where the task is structured, high-volume, latency-sensitive or must be explainable — often both in the same pipeline."*

1.3 Types of learning

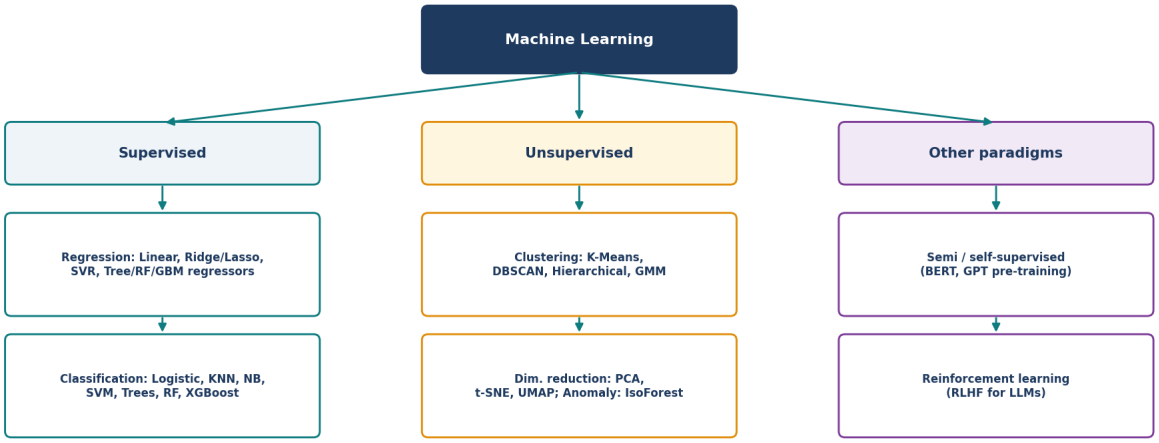


Figure: The ML landscape and where the algorithms in this book belong.

Type	Data	Goal	Examples
Supervised – regression	features X + numeric target y	predict a number	house price, demand, delivery time
Supervised – classification	X + categorical y	predict a class / probability	churn, fraud, spam, loan default
Unsupervised	X only	find structure	customer segments, anomalies, compression
Semi-supervised	few labels + many unlabelled	use unlabelled data	pseudo-labelling
Self-supervised	labels derived from data	learn representations	BERT masked LM, GPT next-token, CLIP
Reinforcement	rewards from actions	learn a policy	games, robotics, RLHF for LLMs

1.4 Key vocabulary

Term	Meaning
Feature (X)	an input variable (age, income, word count)
Target / label (y)	what we predict
Sample / instance	one row
Model / hypothesis	the learned function f(x)
Parameters	learned from data (weights, tree splits)
Hyper-parameters	set before training (max_depth, C, k, learning rate) — tuned with validation
Loss function	error on one sample (MSE, log-loss, hinge)
Cost / objective	average loss + regularisation over the training set
Training / validation / test set	fit parameters / choose hyper-parameters / final unbiased estimate

Term	Meaning
Generalisation	performance on unseen data — the real goal
Overfitting / underfitting	memorising noise / too simple to capture the pattern
Inductive bias	assumptions an algorithm makes (linearity, smoothness, axis-aligned splits)

1.5 The 'No Free Lunch' theorem

No single algorithm is best for every problem. Each algorithm's inductive bias suits some data shapes and fails on others — which is why this book teaches **what each algorithm assumes**, **how it works internally**, and **when to use it**, and why you always compare several models with cross-validation (Chapter 8).

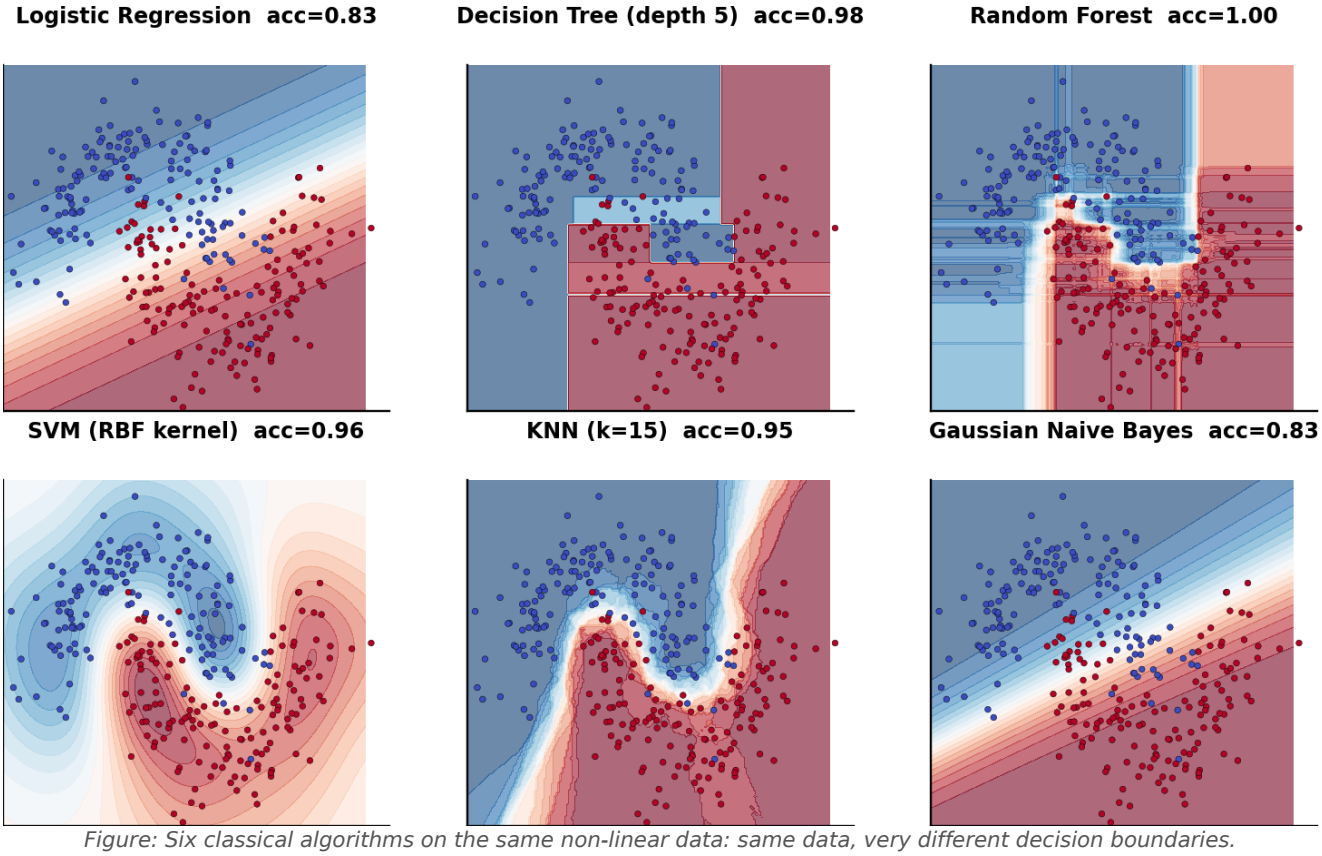


Figure: Six classical algorithms on the same non-linear data: same data, very different decision boundaries.

CHAPTER 2

End-to-End ML Pipeline (Hands-on)

Problem framing • data collection • EDA • cleaning • train/test split • preprocessing with Pipeline & ColumnTransformer • training • evaluation • saving • a complete worked project

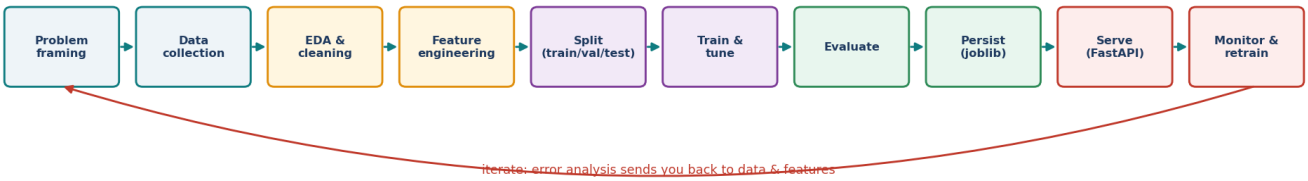


Figure: The ML project life cycle — it is a loop, not a straight line.

2.1 Step 1 — Frame the problem

- Business goal → ML task: 'reduce churn' → binary classification 'will this customer leave in 30 days?'.
- Define the **target** precisely (what counts as churn? over what window?) and the **prediction moment** (what data is available then — avoids leakage).
- Choose the **metric** that reflects cost: recall if missing positives is expensive, precision if false alarms are expensive, RMSE vs MAE for regression.
- Establish a **baseline**: majority class, last value, simple rule. Your model must beat it.

2.2 Step 2 — Data collection & EDA

eda.py

```
import pandas as pd, numpy as np, seaborn as sns, matplotlib.pyplot as plt
df = pd.read_csv("telco_churn.csv")
df.shape, df.info(), df.describe(include="all").T
df.isna().mean().sort_values(ascending=False)          # missing ratio per column
df["Churn"].value_counts(normalize=True)               # class balance (e.g. 73/27)
df.duplicated().sum()
num = df.select_dtypes("number").columns; cat = df.select_dtypes("object").columns.drop("Churn")
df[num].hist(bins=30, figsize=(12, 6)); sns.heatmap(df[num].corr(), annot=True)
sns.boxplot(x="Churn", y="MonthlyCharges", data=df)     # feature vs target
pd.crosstab(df["Contract"], df["Churn"], normalize="index")
```

EDA question	Why
Missing values / types wrong?	decide imputation, fix dtypes (TotalCharges stored as text)
Target distribution?	imbalance → metric choice, stratified split
Distributions, skew, outliers?	transformations, robust scaling, capping
Correlations / multicollinearity?	feature selection, regularisation
Relationship with target?	feature ideas; detect leakage (a feature that is 'too good')
Cardinality of categoricals?	one-hot vs target/ordinal encoding

2.3 Step 3 — Split before you transform

Split **first**, then fit every transformation on the training set only. Use **stratify=y** for classification, time-based splits for temporal data, and group splits if one entity appears in many rows.

LEAKAGE: test statistics influence training **CORRECT: fit on train, apply to test**

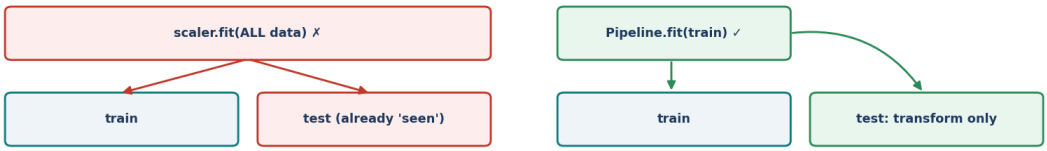


Figure: Data leakage vs the correct way.

2.4 Step 4 — Preprocess with Pipeline & ColumnTransformer

pipeline.py

```
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression

df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
X, y = df.drop(columns=["Churn", "customerID"]), (df["Churn"] == "Yes").astype(int)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)

num = X.select_dtypes("number").columns; cat = X.select_dtypes("object").columns
pre = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median")), ("sc", StandardScaler())]), num),
    ("cat", Pipeline([("imp", SimpleImputer(strategy="most_frequent")), ("oh", OneHotEncoder(handle_unknown="ignore"))]),
        cat)])
model = Pipeline([("pre", pre), ("clf", LogisticRegression(max_iter=1000, class_weight="balanced"))])
model.fit(X_tr, y_tr)                       # imputer, scaler, encoder, model all fit on TRAIN only
proba = model.predict_proba(X_te)[:, 1]
```

Why Pipeline?	
No leakage	transformers are fit inside each CV fold / only on training data
One object	save and deploy preprocessing + model together
Tunable	grid-search pre-processing and model params together: 'clf__C', 'pre__num__imp__strategy'
Clean code	fit / predict on raw DataFrames

2.5 Step 5 — Train, evaluate, iterate

evaluate.py

```
from sklearn.metrics import classification_report, roc_auc_score, confusion_matrix
pred = (proba >= 0.5).astype(int)
print(classification_report(y_te, pred)); print("ROC-AUC", roc_auc_score(y_te, proba)); print(confusion_matrix(y_te,
        pred))
# error analysis: look at the wrong predictions
errs = X_te.assign(y=y_te, p=proba)[(pred != y_te)]; errs.sort_values("p").head(20)
```

2.6 Steps 6–8 — Persist, serve, monitor

Save the fitted pipeline with joblib, serve it behind a FastAPI endpoint, log inputs and predictions, and monitor data drift and performance (Chapter 3). The rest of this book deepens each step: evaluation (3), algorithms (4–7), comparison & tuning (8), bias–variance (9), features (10), ensembles (11) and improvement strategies (12).

CHAPTER 3

Model Evaluation, Persistence & ML as a Service

Train/validation/test • cross-validation strategies • classification metrics • ROC & PR curves • threshold tuning • calibration • regression metrics • saving models (pickle, joblib, ONNX) • FastAPI serving • Docker • monitoring

3.1 Validation strategies



Figure: K-fold cross-validation: every sample is used for validation exactly once.

Strategy	When	sklearn
Hold-out (train/val/test)	large data	train_test_split
K-Fold (k=5/10)	default for small–medium data	KFold, cross_val_score
Stratified K-Fold	classification, imbalance	StratifiedKFold
Group K-Fold	same customer/patient in many rows	GroupKFold
Time-series split	temporal data; never train on the future	TimeSeriesSplit
Leave-One-Out	tiny datasets	LeaveOneOut
Nested CV	unbiased estimate when also tuning hyper-parameters	GridSearchCV inside cross_val_score
Repeated K-Fold	reduce variance of the estimate	RepeatedStratifiedKFold

cross_validation.py

```
from sklearn.model_selection import cross_validate, StratifiedKFold
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
res = cross_validate(model, X, y, cv=cv, scoring=["accuracy", "f1", "roc_auc"], return_train_score=True)
{k: (v.mean().round(3), v.std().round(3)) for k, v in res.items() if "test" in k or "train" in k}
```

3.2 Classification metrics

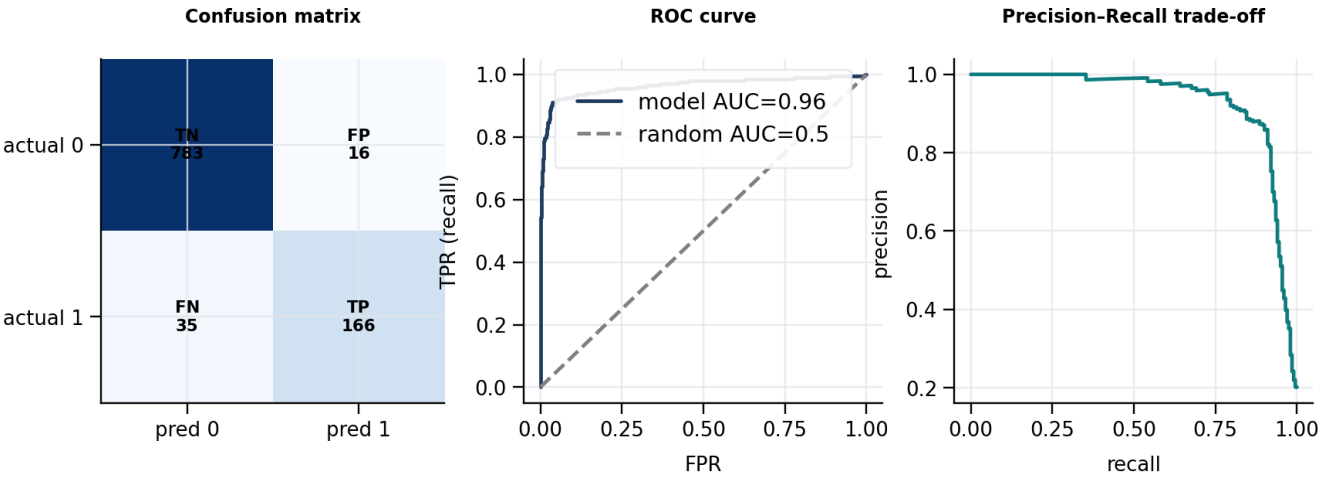


Figure: Confusion matrix, ROC curve and precision–recall curve on an imbalanced dataset.

$$\text{Acc} = \frac{TP + TN}{N} \quad P = \frac{TP}{TP + FP} \quad R = \frac{TP}{TP + FN} \quad \text{Spec} = \frac{TN}{TN + FP} \quad F_\beta = \frac{(1 + \beta^2)PR}{\beta^2 P + R}$$

Metric	Question answered	Use when
Accuracy	fraction correct	balanced classes, equal error costs
Precision	of predicted positives, how many are real?	false positives are costly (spam to inbox, sales-call budget)
Recall / sensitivity	of real positives, how many did we catch?	false negatives are costly (fraud, disease, security)
F1 / Fβ	balance of P and R (β>1 favours recall)	imbalanced data, one number
ROC-AUC	P(random positive scored above random negative)	ranking quality, threshold-free
PR-AUC / Average Precision	precision–recall trade-off over thresholds	rare positives (fraud 0.5%)
Log-loss	quality of predicted probabilities	probabilities used downstream
MCC	correlation between prediction and truth (−1..1)	robust single metric for imbalance
Macro / micro / weighted avg	multi-class aggregation	macro = each class equal; weighted by support

Worked example

Fraud model on 1,000 transactions (50 fraud): TP = 40, FN = 10, FP = 60, TN = 890. Accuracy = 930/1000 = 93%; Precision = 40/100 = 0.40; Recall = 40/50 = 0.80; F1 = 2·0.4·0.8/1.2 = 0.533. A model predicting 'never fraud' scores 95% accuracy with recall 0 — accuracy is misleading on imbalanced data.

Threshold tuning and calibration

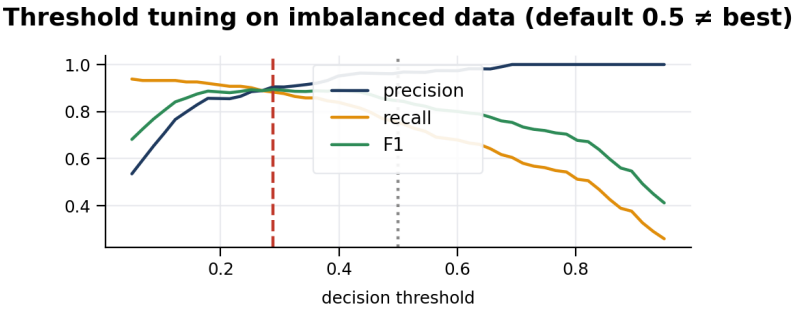


Figure: The default 0.5 threshold is rarely optimal on imbalanced data.

- Choose the threshold on **validation** data to maximise F1 or minimise business cost: cost = c_FP·FP + c_FN·FN.
- **Calibration**: a predicted 0.8 should be right 80% of the time. Check with calibration curves / Brier score; fix with CalibratedClassifierCV (Platt sigmoid or isotonic). Logistic Regression is usually well calibrated; SVM, NB and boosted trees often are not.

3.3 Regression metrics

$$\text{MAE} = \frac{1}{n}\sum|y-\hat{y}| \quad \text{RMSE} = \sqrt{\frac{1}{n}\sum(y-\hat{y})^2} \quad R^2 = 1 - \frac{\sum(y-\hat{y})^2}{\sum(y-\bar{y})^2} \quad R^2_{adj} = 1-(1-R^2)\frac{n-1}{n-p-1}$$

Metric	Notes
MAE	same units as target; robust to outliers; median-optimal
MSE / RMSE	penalises large errors heavily; RMSE in target units
R^2	fraction of variance explained; can be negative on test data
Adjusted R^2	penalises adding useless features
MAPE / sMAPE	percentage error; breaks near y = 0
RMSLE	relative errors; for skewed positive targets (prices)

3.4 Model persistence

Format	How	Notes
pickle	pickle.dump(model, f)	any Python object; security risk when loading untrusted files
joblib	joblib.dump(model, 'm.joblib', compress=3)	preferred for sklearn (efficient for numpy arrays)
ONNX	skl2onnx.convert_sklearn → onnxruntime	language/framework-independent, fast inference
Framework native	xgb.save_model('m.json'), Keras .keras, torch state_dict	version-robust for those libraries
MLflow model	mlflow.sklearn.log_model	packaged with environment + registry versioning

persist.py

```
import joblib, sklearn, json, datetime
joblib.dump(model, "churn_v1.joblib")
json.dump({"sklearn": sklearn.__version__, "trained": str(datetime.date.today()), "features": list(X.columns),
           "metrics": {"roc_auc": 0.84}, "threshold": 0.42}, open("churn_v1.meta.json", "w"))   # model card / metadata
model = joblib.load("churn_v1.joblib")      # load with the SAME library versions
```

Watch out: Never unpickle files from untrusted sources — loading can execute arbitrary code. Pin library versions (requirements.txt) because a model saved with one scikit-learn version may fail or behave differently in another.

3.5 ML as a Service with FastAPI

main.py

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
import joblib, pandas as pd

app = FastAPI(title="Churn Prediction API", version="1.0")
model = joblib.load("churn_v1.joblib"); THRESHOLD = 0.42           # load ONCE at startup

class Customer(BaseModel):
    tenure: int = Field(ge=0); MonthlyCharges: float = Field(gt=0); TotalCharges: float | None = None
    Contract: str; PaymentMethod: str; InternetService: str              # ...all training features
class Prediction(BaseModel):
    churn_probability: float; churn: bool; model_version: str = "v1"

@app.get("/health")
def health(): return {"status": "ok"}

@app.post("/predict", response_model=Prediction)
def predict(c: Customer):
    try: p = float(model.predict_proba(pd.DataFrame([c.model_dump()]))[0, 1])
    except Exception as e: raise HTTPException(status_code=422, detail=str(e))
    return Prediction(churn_probability=round(p, 4), churn=p >= THRESHOLD)

@app.post("/predict_batch")
def predict_batch(cs: list[Customer]):
    p = model.predict_proba(pd.DataFrame([c.model_dump() for c in cs]))[:, 1]
    return [{"churn_probability": float(v), "churn": bool(v >= THRESHOLD)} for v in p]
# run: uvicorn main:app --reload   → Swagger UI at /docs
```

test_api.py

```
from fastapi.testclient import TestClient
from main import app
client = TestClient(app)
def test_health(): assert client.get("/health").json() == {"status": "ok"}
def test_predict():
    r = client.post("/predict", json={"tenure": 2, "MonthlyCharges": 90.5, "Contract": "Month-to-month", "PaymentMethod":
        "Electronic check", "InternetService": "Fiber optic"})
    assert r.status_code == 200 and 0 <= r.json()["churn_probability"] <= 1
```

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY main.py churn_v1.joblib ./
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Serving pattern	When
Real-time REST API	per-request decisions (fraud check, routing)
Batch scoring	nightly scoring of all customers for a campaign
Streaming	events from Kafka scored continuously
Embedded / edge	ONNX in an app or device

3.6 Monitoring after deployment

- **Data drift**: input distributions change (PSI, KS test per feature).
- **Concept drift**: the relationship X → y changes (fraud tactics evolve) — detected via performance on newly labelled data.
- **Prediction drift**: share of positives changes suddenly.
- **System**: latency, errors, throughput.
- Retrain on schedule or when drift/performance triggers fire; validate the new model against the current one before switching.

CHAPTER 4

Decision Trees: Algorithm & Implementation

Intuition • terminology • impurity (Gini, entropy) • information gain • CART algorithm step by step • worked numeric example • regression trees • stopping & pruning • hyper-parameters • from-scratch implementation • sklearn • visualising • pros/cons • when to use

4.1 Intuition

A decision tree learns a sequence of **if–else questions** on the features that split the data into increasingly **pure** groups. To predict, a sample travels from the root down the branches according to its feature values and receives the majority class (classification) or mean value (regression) of the leaf it lands in. It mimics how humans make decisions: '*Is income > 50k? If yes, is credit history good? …*'

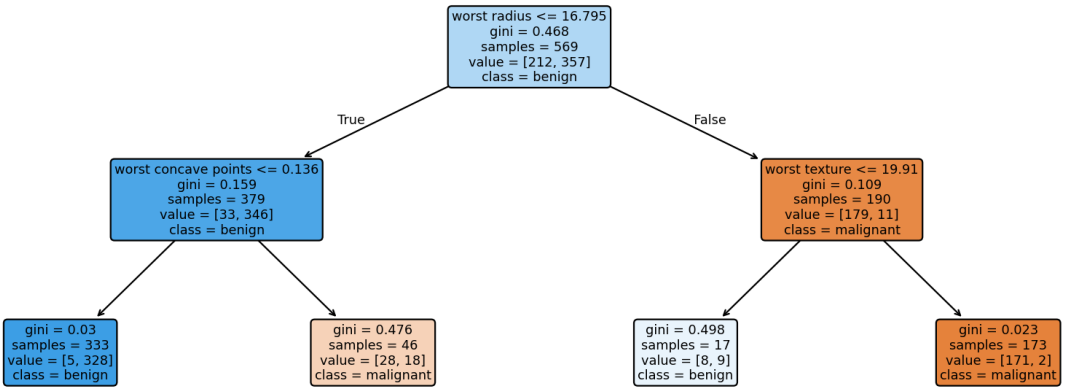


Figure: A real depth-2 tree trained on the breast-cancer dataset (sklearn plot_tree).

Term	Meaning
Root node	first split, on all data
Internal / decision node	a test on one feature: x_j ≤ threshold
Branch	outcome of a test
Leaf / terminal node	final prediction
Depth	longest path from root to a leaf
Pure node	contains only one class (impurity 0)
Split criterion	how good a split is (Gini, entropy, MSE)

4.2 Measuring impurity

$$\text{Gini}(S) = 1 - \sum_{k=1}^{K} p_k^2 \qquad \text{Entropy}(S) = -\sum_{k=1}^{K} p_k \log_2 p_k \qquad \text{MSE}(S) = \frac{1}{|S|}\sum_{i \in S}(y_i - \bar{y}_S)^2$$

Impurity is maximal at 50/50, zero when pure

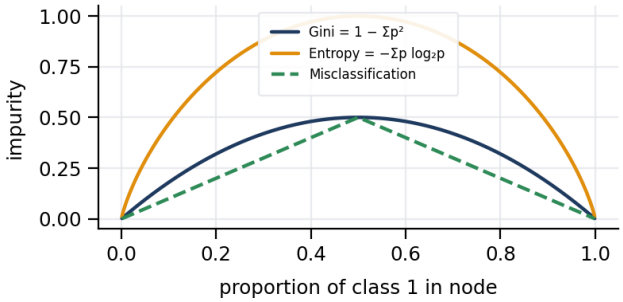


Figure: Impurity measures for a binary node.

$$\text{Gain}(S, \text{split}) = I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R)$$

information gain when I = entropy; Gini decrease when I = Gini

4.3 The CART algorithm (Classification And Regression Trees)

1. Start with all training samples at the root.
2. For **every feature** j and **every candidate threshold** t (midpoints between sorted unique values), split into $S_L = \{x_j \le t\}$ and $S_R = \{x_j > t\}$.
3. Compute the weighted impurity of the children; choose the (j, t) with the **largest impurity decrease** (greedy — best now, not globally optimal).
4. Recurse on each child.
5. Stop when a node is pure, or max_depth / min_samples_split / min_samples_leaf / min_impurity_decrease is reached.
6. Leaf prediction: majority class (with class proportions as probabilities) or mean of y.

Complexity: sorting gives roughly O(d · n log n) per level. CART always makes **binary** splits. Other algorithms: **ID3** (entropy, categorical features, multi-way splits), **C4.5** (gain ratio, handles continuous features and missing values, pruning), **CHAID** (chi-square tests).

4.4 Worked example — choosing the root split

#	Income	Owns house	Approved?
1	High	Yes	Yes
2	High	No	Yes
3	High	Yes	Yes
4	Low	Yes	Yes
5	Low	No	No
6	Low	No	No
7	High	No	No

#	Income	Owns house	Approved?
8	Low	Yes	Yes
9	Low	No	No
10	High	Yes	Yes

Root: 6 Yes / 4 No → Gini = $1 - (0.6^2 + 0.4^2)$ = **0.48**; Entropy = $-0.6 \log_2 0.6 - 0.4 \log_2 0.4$ = **0.971**.

Split	Left child	Right child	Weighted Gini	Gini decrease	Info gain
Income = High?	High: 4 Yes / 1 No → Gini 0.32, H 0.722	Low: 2 Yes / 3 No → Gini 0.48, H 0.971	0.5·0.32 + 0.5·0.48 = 0.40	0.08	0.971 − 0.846 = 0.125
Owns house = Yes?	Yes: 5 Yes / 0 No → Gini 0, H 0	No: 1 Yes / 4 No → Gini 0.32, H 0.722	0.5·0 + 0.5·0.32 = 0.16	**0.32**	**0.610**

'Owns house' wins under both criteria → root split. The left child is pure (leaf: Approved). Recurse on the right child (1 Yes / 4 No) using Income: High → 1 Yes/1 No, Low → 0 Yes/3 No; and so on.

4.5 Regression trees

Same algorithm, but impurity = **variance (MSE)** of y in a node and the leaf predicts the **mean**. The result is a piecewise-constant function — it cannot extrapolate beyond the training range.

Regression tree = piecewise-constant prediction (leaf mean)

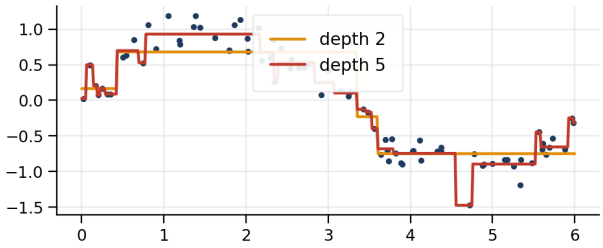


Figure: Deeper regression trees fit finer steps — and eventually the noise.

4.6 Overfitting, stopping and pruning

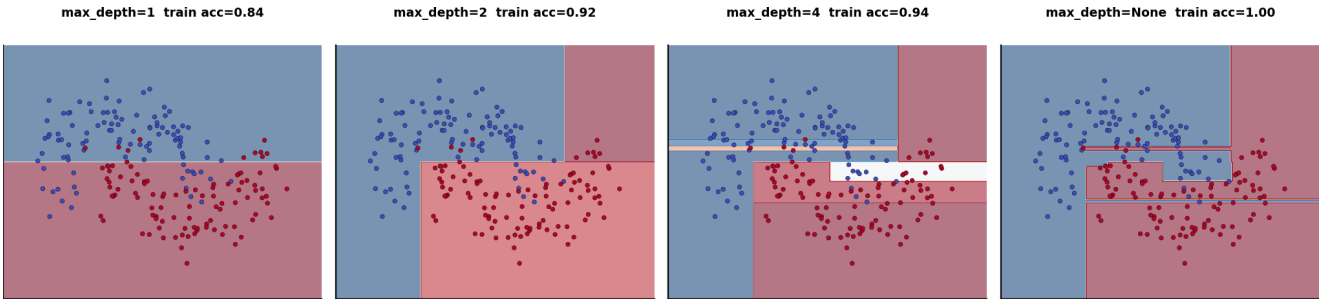


Figure: As depth grows, training accuracy → 1.0 and the boundary becomes jagged (overfitting).

Hyper-parameter (sklearn)	Effect	Typical
max_depth	limits tree depth	3–10 (single tree)
min_samples_split	min samples to split a node	2–50
min_samples_leaf	min samples in each leaf — smooths predictions	1–50 (or 1–5% of n)
max_leaf_nodes	caps number of leaves (best-first growth)	—
max_features	features considered per split (randomness)	None for one tree; 'sqrt' in RF

Hyper-parameter (sklearn)	Effect	Typical
min_impurity_decrease	require minimum gain	small values
criterion	'gini', 'entropy', 'log_loss' / 'squared_error', 'absolute_error', 'poisson'	gini / squared_error
class_weight	'balanced' for imbalance	—
ccp_alpha	cost-complexity (post-)pruning strength	choose by CV

$$R_\alpha(T) = R(T) + \alpha\,|\text{leaves}(T)|$$

minimal cost-complexity pruning: grow a full tree, then prune subtrees whose error reduction is not worth their leaves

pruning.py

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score
path = DecisionTreeClassifier(random_state=0).cost_complexity_pruning_path(X_tr, y_tr)
scores = [(a, cross_val_score(DecisionTreeClassifier(ccp_alpha=a, random_state=0), X_tr, y_tr, cv=5).mean()) for a in
        path.ccp_alphas[::5]]
best_alpha = max(scores, key=lambda s: s[1])[0]
```

4.7 Implementation from scratch

decision_tree_from_scratch.py

```
import numpy as np
class Node:
    def __init__(self, feature=None, thr=None, left=None, right=None, value=None):
        self.feature, self.thr, self.left, self.right, self.value = feature, thr, left, right, value

def gini(y):
    _, c = np.unique(y, return_counts=True); p = c / c.sum(); return 1 - np.sum(p ** 2)

class MyDecisionTree:
    def __init__(self, max_depth=5, min_samples_split=2): self.max_depth, self.min_split = max_depth, min_samples_split
    def fit(self, X, y): self.root = self._grow(np.asarray(X), np.asarray(y), 0); return self

    def _best_split(self, X, y):
        best, parent = (None, None, 0.0), gini(y)
        for j in range(X.shape[1]):
            vals = np.unique(X[:, j]); thrs = (vals[:-1] + vals[1:]) / 2          # midpoints
            for t in thrs:
                m = X[:, j] <= t; nl, nr = m.sum(), (~m).sum()
                gain = parent - (nl * gini(y[m]) + nr * gini(y[~m])) / len(y)
                if gain > best[2]: best = (j, t, gain)
        return best

    def _grow(self, X, y, depth):
        if depth >= self.max_depth or len(y) < self.min_split or len(np.unique(y)) == 1:
            return Node(value=np.bincount(y).argmax())                            # leaf: majority class
        j, t, gain = self._best_split(X, y)
        if j is None: return Node(value=np.bincount(y).argmax())
        m = X[:, j] <= t
        return Node(j, t, self._grow(X[m], y[m], depth + 1), self._grow(X[~m], y[~m], depth + 1))

    def _predict_one(self, x, n):
        while n.value is None: n = n.left if x[n.feature] <= n.thr else n.right
        return n.value
    def predict(self, X): return np.array([self._predict_one(x, self.root) for x in np.asarray(X)])

from sklearn.datasets import load_iris
X, y = load_iris(return_X_y=True)
print((MyDecisionTree(max_depth=3).fit(X, y).predict(X) == y).mean())         # ≈ 0.97
```

4.8 Decision trees in scikit-learn

sklearn_tree.py

```
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor, plot_tree, export_text
from sklearn.model_selection import GridSearchCV
grid = GridSearchCV(DecisionTreeClassifier(random_state=42),
                    {"max_depth": [3, 4, 5, 6, 8, None], "min_samples_leaf": [1, 5, 10, 20], "criterion": ["gini",
        "entropy"]},
                    cv=5, scoring="f1")
grid.fit(X_tr, y_tr); tree = grid.best_estimator_
print(export_text(tree, feature_names=list(feature_names)))     # human-readable rules
plot_tree(tree, feature_names=feature_names, filled=True, max_depth=3)
tree.feature_importances_                                        # total impurity decrease per feature
```

4.9 Strengths, weaknesses and when to use

Strengths	Weaknesses
Highly interpretable (rules, visual); no scaling needed; handles numeric & categorical, non-linear relations and interactions; fast prediction; feature importance; little preprocessing	High variance / unstable (small data change → different tree); overfits without constraints; greedy, axis-aligned splits (struggles with diagonal boundaries); piecewise-constant (no extrapolation); biased toward features with many split points

Use a single decision tree when…	Prefer something else when…
you must explain decisions as rules (credit policy, medical triage, compliance); as a quick baseline / EDA tool to discover interactions; data is small and clean	accuracy matters most → Random Forest / Gradient Boosting; smooth or linear relationships → linear models; extrapolation needed → linear/GBM with care

CHAPTER 5

Random Forest: an Ensemble of Trees

Wisdom of crowds • bagging & bootstrap • feature randomness • algorithm • why it reduces variance (math) • OOB error • feature importance (impurity vs permutation) • hyper-parameters • Extra Trees • implementation • when to use

5.1 Intuition

One deep tree has low bias but high variance. If we train **many different trees** and **average** them, individual errors cancel out. Random Forest (Breiman, 2001) makes the trees different in two ways: each tree sees a different **bootstrap sample** of rows, and each split considers only a **random subset of features**.

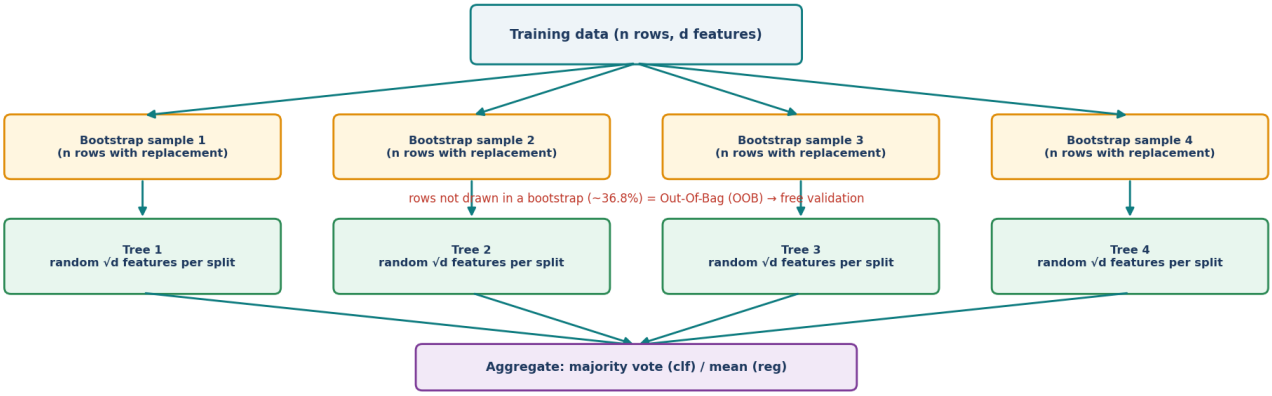


Figure: Random Forest = bagging + random feature subsets + aggregation.

5.2 The algorithm

1. For b = 1 … B (n_estimators): draw a bootstrap sample (n rows **with replacement**).
2. Grow a decision tree on it, usually deep (no pruning); at **each split** choose the best split among only m randomly chosen features (m = √d for classification, d/3 or all for regression in many implementations).
3. Prediction: classification = majority vote (or average of class probabilities — sklearn averages probabilities); regression = mean of tree outputs.

5.3 Why averaging works (the math)

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B} T_b\right) = \rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2$$

σ² = variance of one tree, ρ = correlation between trees. As B grows the second term vanishes, but ρσ² remains — so we must make trees **less correlated**. That is exactly what random feature selection does: without it, every tree would split on the same dominant feature first. Bias stays about the same as one deep tree; variance drops a lot.

Bootstrap and out-of-bag (OOB) samples

$$P(\text{row not chosen in } n \text{ draws}) = \left(1 - \frac{1}{n}\right)^n \longrightarrow e^{-1} \approx 0.368$$

Each tree never sees ~36.8% of rows. Predicting each row using only trees that did not see it gives the **OOB score** — an almost free cross-validation estimate (oob_score=True).

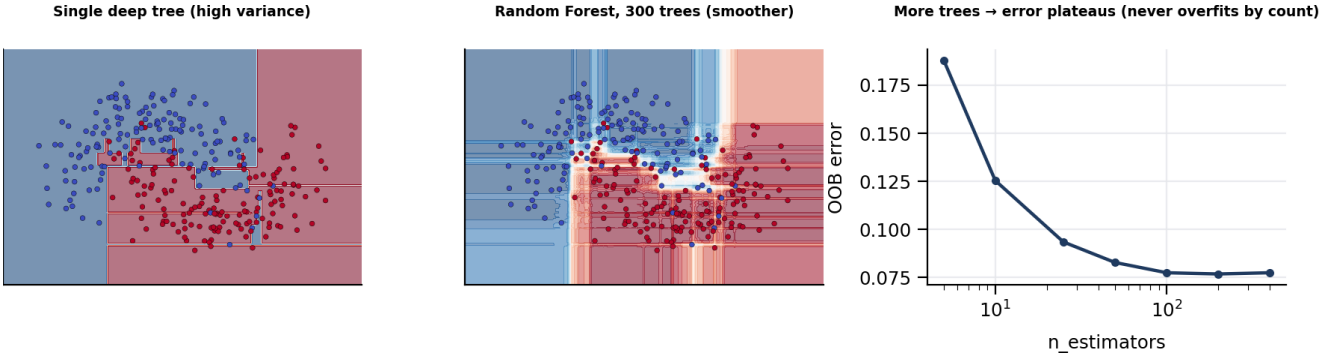


Figure: Left/middle: a single tree vs a forest on noisy data. Right: OOB error falls and plateaus as trees are added (computed).

5.4 Feature importance

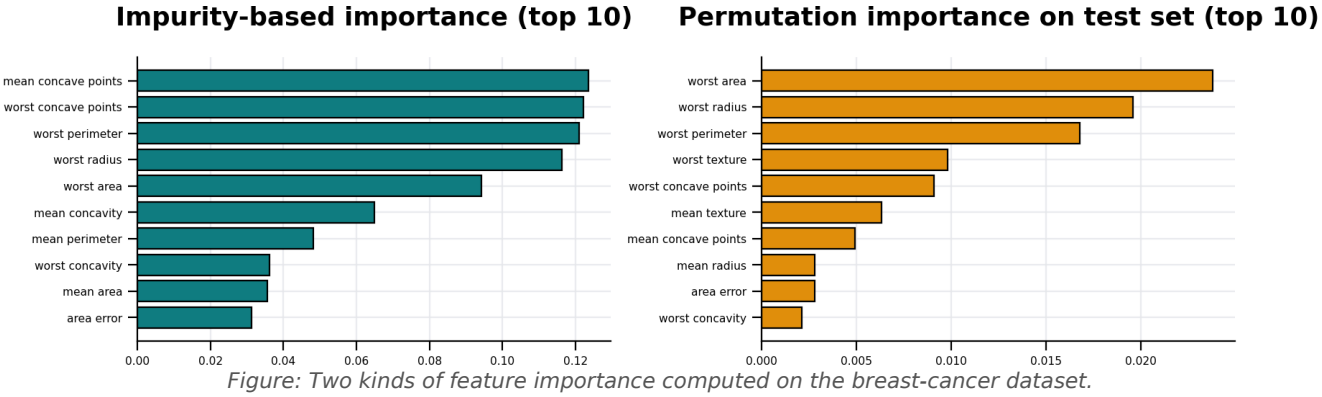


Figure: Two kinds of feature importance computed on the breast-cancer dataset.

Method	How	Caveat
Impurity-based (MDI) feature_importances_	total impurity decrease from splits on the feature, averaged over trees	computed on training data; biased toward high-cardinality / continuous features
Permutation importance	shuffle a feature on validation data and measure the score drop	reliable; slower; correlated features share/hide importance
SHAP values	game-theoretic contribution per prediction (TreeSHAP is fast)	best for local + global explanations

5.5 Hyper-parameters

Parameter	Effect	Guidance
n_estimators	number of trees; more = more stable, never overfits by itself, slower	200–1000; stop when OOB plateaus
max_features	features per split; lower = more diversity, more bias	'sqrt' (clf), 1.0 or 0.33 (reg); tune
max_depth / min_samples_leaf	tree complexity	often None / 1 works; increase leaf size for noisy data
bootstrap / max_samples	row sampling	True; max_samples < 1 speeds up
class_weight	'balanced' / 'balanced_subsample'	imbalanced data
oob_score	free validation estimate	True
n_jobs	parallel trees	-1

random_forest.py

```
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor, ExtraTreesClassifier
from sklearn.inspection import permutation_importance
rf = RandomForestClassifier(n_estimators=500, max_features="sqrt", min_samples_leaf=2, class_weight="balanced",
                            oob_score=True, n_jobs=-1, random_state=42).fit(X_tr, y_tr)
print("OOB:", rf.oob_score_, "test:", rf.score(X_te, y_te))
pi = permutation_importance(rf, X_te, y_te, n_repeats=10, random_state=0)
sorted(zip(pi.importances_mean, feature_names), reverse=True)[:10]
```

5.6 Random Forest from scratch (using our tree)

random_forest_from_scratch.py

```
class MyRandomForest:
    def __init__(self, n_trees=50, max_depth=8, max_features="sqrt"): self.n, self.depth, self.mf = n_trees, max_depth,
        max_features
    def fit(self, X, y):
        X, y = np.asarray(X), np.asarray(y); n, d = X.shape; k = int(np.sqrt(d)) if self.mf == "sqrt" else d
        self.trees = []
        for _ in range(self.n):
            rows = np.random.randint(0, n, n)                       # bootstrap
            cols = np.random.choice(d, k, replace=False)            # (simplified: per tree instead of per split)
            t = MyDecisionTree(max_depth=self.depth).fit(X[rows][:, cols], y[rows]); self.trees.append((t, cols))
        return self
    def predict(self, X):
        X = np.asarray(X); votes = np.array([t.predict(X[:, c]) for t, c in self.trees])
        return np.array([np.bincount(v).argmax() for v in votes.T])  # majority vote
```

(The real algorithm samples features at **every split**; the simplified version samples per tree — 'random subspace' method — but shows the idea.)

5.7 Extra Trees and other variants

Variant	Difference
Extra Trees (Extremely Randomized Trees)	thresholds chosen at random (best of random ones), usually no bootstrap → even less variance, faster; sometimes slightly more bias
Balanced Random Forest (imblearn)	undersamples the majority class in each bootstrap
Isolation Forest	random trees for anomaly detection — anomalies are isolated in fewer splits
Quantile Regression Forest	predicts intervals, not just the mean

5.8 When to use Random Forest

Great for	Not ideal for
strong default for tabular classification/regression with little tuning; non-linear interactions; mixed feature types; robustness to outliers and noisy features; feature importance; parallel training	very high-dimensional sparse text (linear models better); extrapolation (trend beyond training range); strict latency/memory limits (hundreds of deep trees); when the last % of accuracy matters (tuned gradient boosting usually wins); need a single simple explanation

CHAPTER 6

Support Vector Machines: Theory & Kernels

Maximum-margin intuition • hard-margin math • support vectors • soft margin & C • hinge loss • dual problem • kernel trick • linear, polynomial, RBF, sigmoid kernels • γ • multi-class • SVR • scaling • tuning • when to use

6.1 Intuition: the widest street

Many lines can separate two classes. SVM picks the one with the **maximum margin** — the widest 'street' between the classes — because a wider margin generalises better. Only the points on the edge of the street (**support vectors**) determine the boundary; all other points could be removed without changing it.

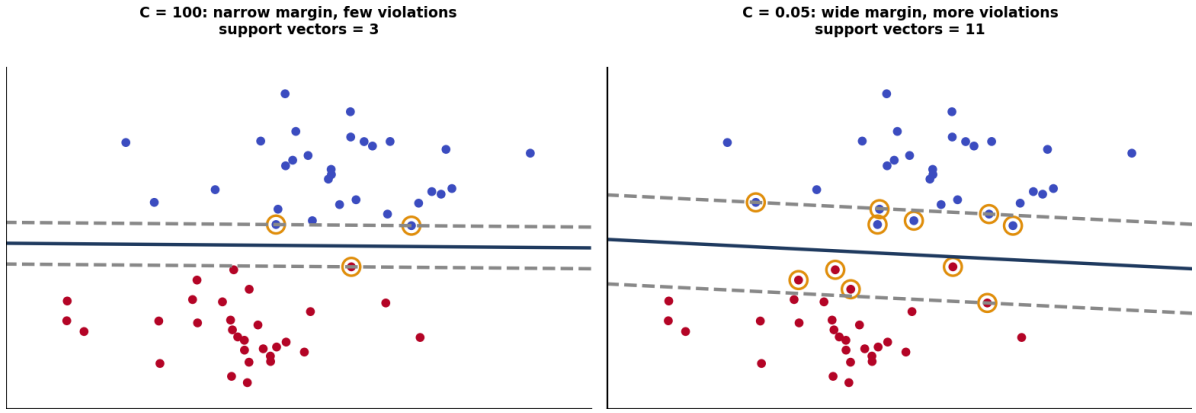


Figure: Linear SVM: solid line = decision boundary, dashed = margins, circled = support vectors. C controls the trade-off.

6.2 Hard-margin SVM (separable data)

$$f(x) = w^T x + b, \qquad \hat{y} = \text{sign}(f(x)), \qquad \text{margin} = \frac{2}{\|w\|}$$

$$\min_{w,\,b} \frac{1}{2}\|w\|^2 \quad \text{subject to} \quad y_i(w^T x_i + b) \geq 1 \;\; \forall i, \qquad y_i \in \{-1, +1\}$$

Maximising 2/‖w‖ is the same as minimising ½‖w‖² (a convex quadratic program → one global optimum). The constraints require every point to be on the correct side, at least 1 unit (in function value) from the boundary.

6.3 Soft margin, C and hinge loss

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i \quad \text{s.t.} \quad y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

$$\iff \quad \min_{w,b} \frac{1}{2}\|w\|^2 + C\sum_i \max(0,\ 1 - y_i(w^T x_i + b)) \quad \text{(hinge loss)}$$

Slack ξ_i	Meaning
0	correct and outside the margin
$0 < \xi < 1$	correct but inside the margin
$\xi > 1$	misclassified

C	Effect
large C	violations are expensive → narrow margin, fits training data closely → low bias, high variance (overfit risk)
small C	violations are cheap → wide margin, more regularisation → higher bias, lower variance

6.4 The dual problem and why it matters

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2}\sum_i\sum_j \alpha_i\alpha_j y_i y_j x_i^T x_j \quad \text{s.t.} \quad 0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i y_i = 0$$

$$f(x) = \sum_{i \in SV} \alpha_i y_i x_i^T x + b$$

Only support vectors have $\alpha_i > 0$. Crucially, the data appear **only through dot products** $x_i^T x_j$ — which lets us replace them by a kernel.

6.5 The kernel trick

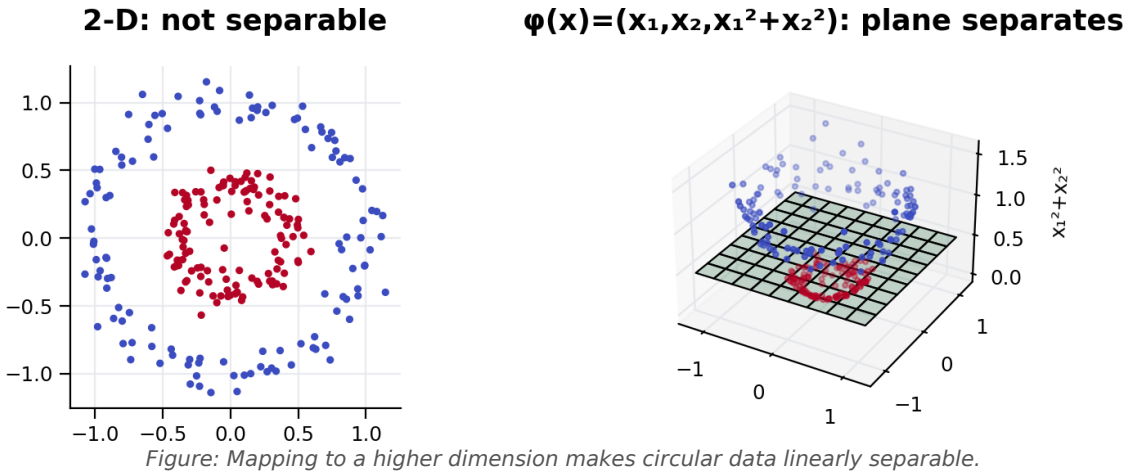


Figure: Mapping to a higher dimension makes circular data linearly separable.

$$K(x, x') = \phi(x)^T\phi(x') \qquad \Rightarrow \qquad f(x) = \sum_{i \in SV} \alpha_i y_i K(x_i, x) + b$$

A kernel computes the dot product in a (possibly infinite-dimensional) feature space φ **without ever computing φ(x)**. Example: for $x \in \mathbb{R}^2$, $K(x, z) = (x^T z)^2$ equals $\varphi(x)^T\varphi(z)$ with $\varphi(x) = (x_1^2,$

$\sqrt{2}\cdot x_1 x_2$, x_2^2). A valid kernel must be symmetric positive semi-definite (Mercer's condition).

Kernel	Formula	Parameters	Use
Linear	$x^T x'$	C	high-dimensional sparse data (text), many features vs samples, baseline
Polynomial	$(\gamma \cdot x^T x' + r)^d$	degree d, γ, coef0 r, C	feature interactions; image processing; rarely best
RBF / Gaussian	$\exp(-\gamma \lVert x - x' \rVert^2)$	γ, C	default non-linear kernel; smooth, local similarity
Sigmoid	$\tanh(\gamma \cdot x^T x' + r)$	γ, r, C	neural-net-like; not always PSD; rarely used
Custom	string, graph kernels	—	special structures

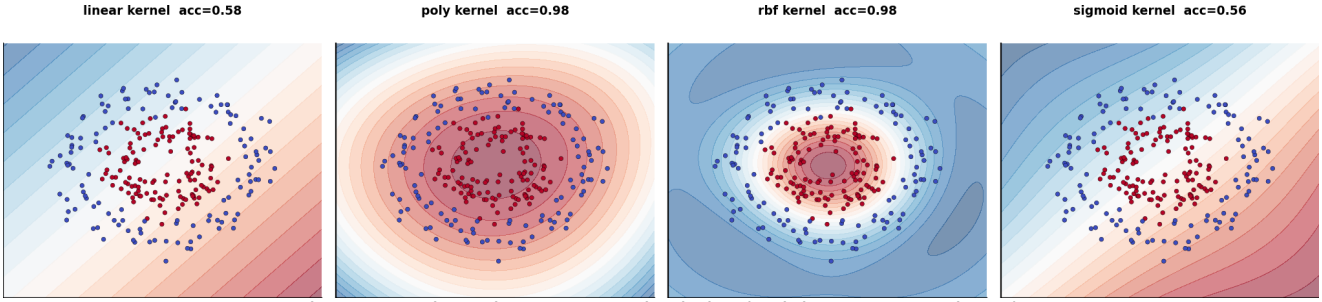


Figure: Four kernels on concentric circles (training accuracy shown).

γ (gamma) in the RBF kernel

γ controls how far one training point's influence reaches: **small γ** → each point influences a wide area → smooth, nearly linear boundary (underfit); **large γ** → influence only very close → islands around points (overfit). sklearn default gamma='scale' = 1 / (n_features · X.var()). Tune C and γ **together** on a log grid.

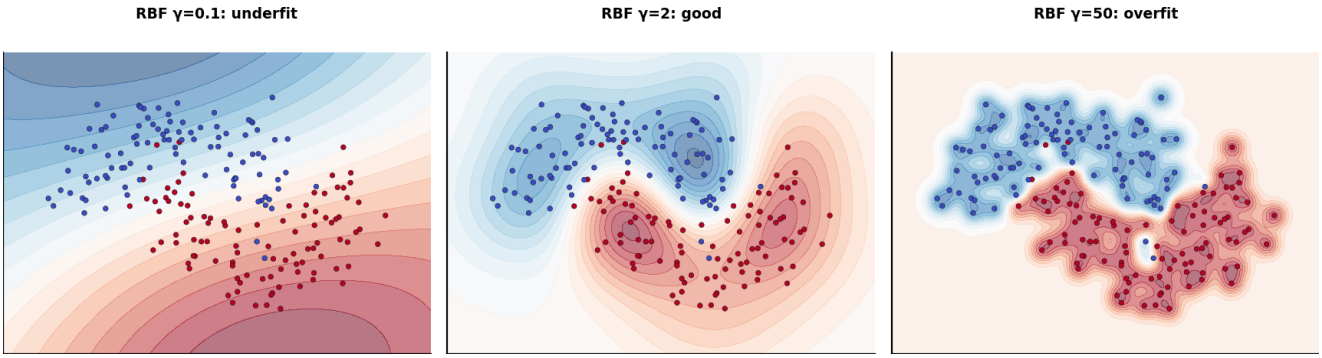


Figure: Effect of γ on the RBF decision boundary.

6.6 Practical details

- **Always scale features** (StandardScaler) — SVM uses distances/dot products; unscaled large-range features dominate.
- **Multi-class**: SVC uses one-vs-one (K(K−1)/2 classifiers); LinearSVC uses one-vs-rest.
- **Probabilities**: SVMs output distances, not probabilities; SVC(probability=True) uses Platt scaling with internal CV (slower).
- **Imbalance**: class_weight='balanced'.
- **Scalability**: kernel SVC training is between $O(n^2)$ and $O(n^3)$ and memory heavy → fine up to ~10^4–10^5 rows; for more use LinearSVC / SGDClassifier(loss='hinge') or kernel approximations (Nystroem, RBFSampler).

- **SVR** (regression): fits a tube of width ε; points inside the tube give zero loss; C and γ as before.

$$\text{SVR}: \ \min \frac{1}{2}\|w\|^2 + C\sum_i \max(0,\ |y_i - f(x_i)| - \varepsilon)$$

svm.py

```
from sklearn.svm import SVC, LinearSVC, SVR
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV
svm = make_pipeline(StandardScaler(), SVC(kernel="rbf", class_weight="balanced"))
grid = GridSearchCV(svm, {"svc__C": [0.1, 1, 10, 100], "svc__gamma": ["scale", 0.001, 0.01, 0.1, 1]}, cv=5, scoring="f1",
        n_jobs=-1)
grid.fit(X_tr, y_tr); print(grid.best_params_, grid.best_score_)
grid.best_estimator_.named_steps["svc"].support_vectors_.shape     # how many support vectors
text_clf = make_pipeline(TfidfVectorizer(ngram_range=(1, 2)), LinearSVC(C=1.0))   # linear SVM: text classification
        workhorse
```

6.7 When to use SVM

Use SVM when	Avoid when
small-to-medium datasets (≤ ~50k rows) with clear margins; high-dimensional data (text, genomics) — linear SVM; non-linear boundaries with RBF and well-scaled features; robustness to outliers via soft margin	very large datasets (kernel SVM too slow); you need calibrated probabilities or easy interpretation; lots of noise / overlapping classes; many categorical features (trees easier); you don't have time to tune C and γ

CHAPTER 7

Classical ML Models Overview & When to Use Them

Linear Regression • Ridge / Lasso / Elastic Net • Polynomial regression • Logistic Regression • KNN • Naive Bayes • (Trees, RF, SVM recap) • K-Means • DBSCAN • Hierarchical • GMM • Isolation Forest • master comparison table • scenario → algorithm guide

For every algorithm: **what it is**, **how it works** (math), **key hyper-parameters**, **assumptions**, **pros & cons**, and **when to use it** with real scenarios.

7.1 Linear Regression

$$\hat{y} = w_0 + w_1 x_1 + \cdots + w_d x_d = w^T x + b \qquad J(w) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \qquad w^* = (X^T X)^{-1} X^T y$$

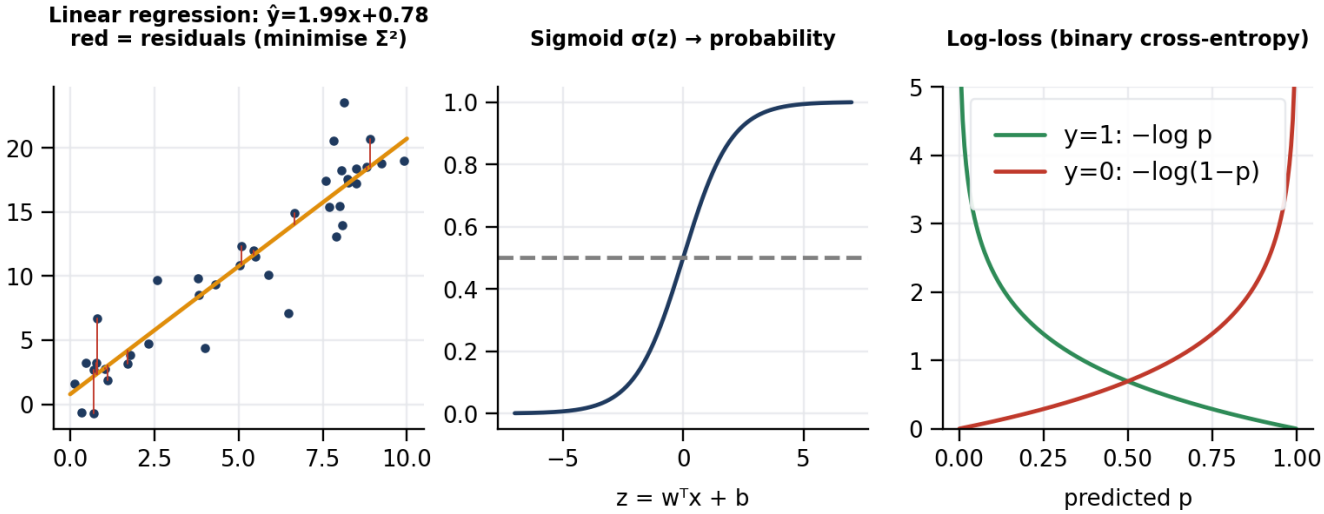


Figure: Linear regression residuals, the sigmoid used by logistic regression, and log-loss.

- **How it learns**: closed-form normal equation (small d) or gradient descent: w ← w − η·(2/n)Xᵀ(Xw − y).
- **Assumptions**: linear relationship, independent errors, homoscedasticity (constant error variance), normally distributed residuals (for inference), little multicollinearity (check VIF).
- **Interpretation**: w_j = expected change in y for +1 unit of x_j holding others constant (standardise to compare).
- **Hyper-parameters**: none (fit_intercept); regularised versions add α.

When to use	Scenarios	Watch out
target is continuous and roughly linear in features; need interpretability; baseline for any regression	sales vs ad spend, salary vs experience, price elasticity, quick forecasting baselines	outliers (use Huber), non-linearity (add features / use trees), multicollinearity (use Ridge)

Polynomial regression

Add powers and interactions of features (PolynomialFeatures(degree=2)) and fit a linear model on them — still linear in the parameters, but curved in x. High degrees overfit → combine with Ridge.

Ridge, Lasso and Elastic Net (regularised regression)

$$\text{Ridge}: J + \alpha\|w\|_2^2 \qquad \text{Lasso}: J + \alpha\|w\|_1 \qquad \text{ElasticNet}: J + \alpha\left(\rho\|w\|_1 + \frac{1-\rho}{2}\|w\|_2^2\right)$$

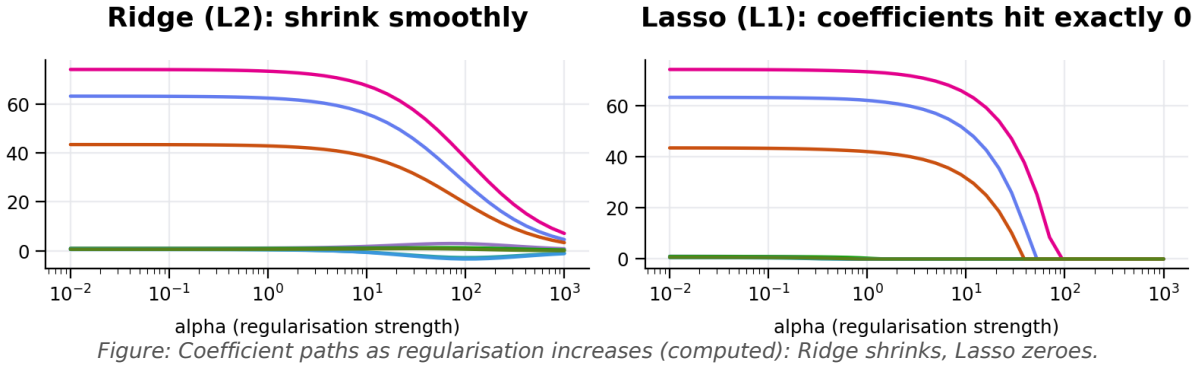


Figure: Coefficient paths as regularisation increases (computed): Ridge shrinks, Lasso zeroes.

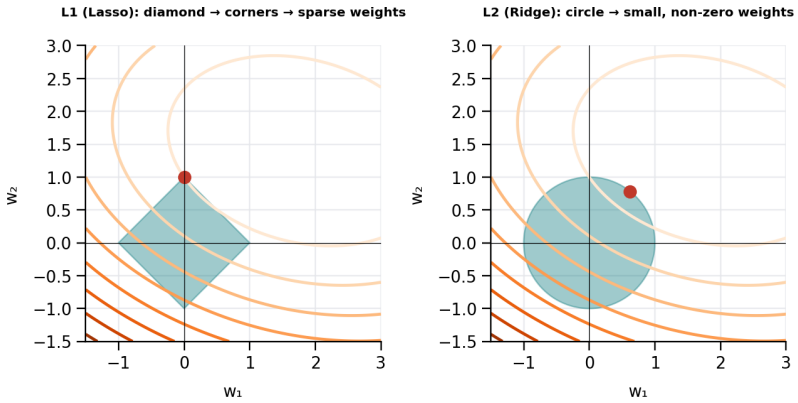


Figure: Why L1 gives sparse solutions: the diamond's corners lie on the axes.

Model	Use when
Ridge (L2)	many correlated features; keep all features but shrink; multicollinearity
Lasso (L1)	you suspect few features matter → automatic feature selection
Elastic Net	many correlated features AND want sparsity (Lasso picks one of a correlated group arbitrarily)

7.2 Logistic Regression

$$P(y=1 \mid x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}} \qquad \log\frac{p}{1-p} = w^T x + b$$

$$J(w) = -\frac{1}{n}\sum_i [y_i \log p_i + (1-y_i)\log(1-p_i)] + \frac{1}{2C}\|w\|^2 \qquad \nabla_w J = \frac{1}{n} X^T(p - y) + \frac{w}{C}$$

- A **linear classifier** that models the log-odds; output is a calibrated probability; decision boundary $w^T x + b = 0$ is a hyperplane.
- e^{w_j} = odds ratio: +1 unit of x_j multiplies the odds by e^{w_j}.
- Multi-class: one-vs-rest or multinomial (softmax) — sklearn handles both.

- Hyper-parameters: **C** (inverse regularisation strength), penalty ('l2', 'l1', 'elasticnet'), solver ('lbfgs', 'liblinear', 'saga'), class_weight.
- Needs scaled features for regularisation to be fair.

When to use	Scenarios	Limitations
binary/multi-class baseline; need probabilities and explanations (regulated domains); high-dimensional sparse data	credit scoring scorecards, churn, click-through rate, medical risk, text classification on TF-IDF, classifier on LLM embeddings	linear boundary only (engineer features or use trees/SVM-RBF); sensitive to outliers and correlated features (regularise)

7.3 K-Nearest Neighbours (KNN)

$$\hat{y}(x) = \text{mode}\{y_i : \ i \in N_k(x)\} \ \ \text{(clf)}, \qquad \hat{y}(x) = \frac{1}{k} \sum_{i \in N_k(x)} y_i \ \ \text{(reg)}, \qquad d_p(x, z) = \left(\sum_j |x_j - z_j|^p \right)^{1/p}$$

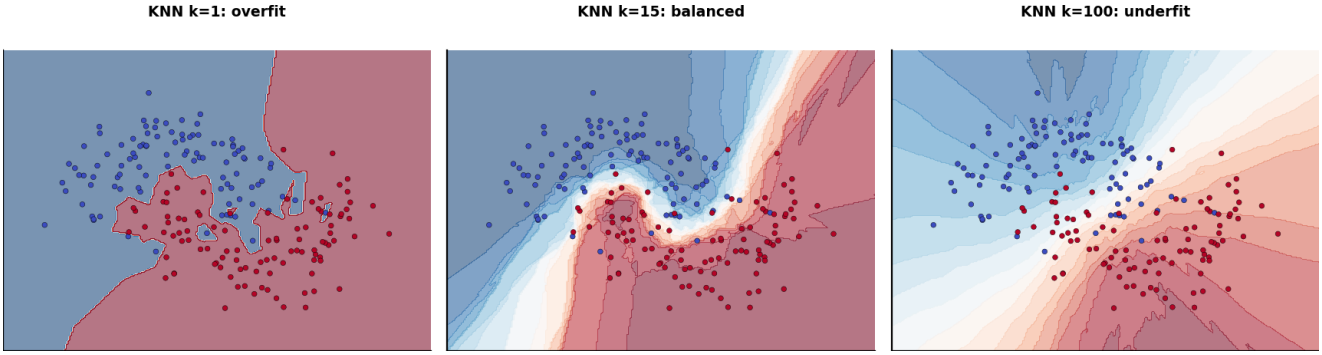


Figure: k controls smoothness: k=1 memorises, very large k underfits.

- **Lazy learner**: 'training' just stores data; all work happens at prediction (O(n·d) per query without indexing).
- Hyper-parameters: **k** (odd for binary; tune by CV), distance metric (Euclidean p=2, Manhattan p=1, cosine for text/embeddings), weights='distance'.
- Must scale features; suffers from the **curse of dimensionality** (distances become similar in high dimensions); speed-ups: KD-tree, Ball-tree, approximate NN (FAISS, HNSW).

When to use	Scenarios	Avoid
small/medium data, low dimension, irregular boundaries; similarity-based reasoning; recommendation 'users like you'	recommendation, case-based reasoning, imputation (KNNImputer), anomaly detection, **semantic search in RAG (kNN over embeddings)**	large data with low latency (unless ANN index), many irrelevant features, unscaled data

7.4 Naive Bayes

$$P(c \mid x) \propto P(c) \prod_{j=1}^{d} P(x_j \mid c), \qquad \hat{c} = \arg\max_c \ \log P(c) + \sum_j \log P(x_j \mid c)$$

Variant	Likelihood $P(x_j \mid c)$	Data
GaussianNB	Normal(μ_{jc}, σ^2_{jc})	continuous features
MultinomialNB	word-count frequencies with Laplace smoothing α	text counts / TF-IDF
BernoulliNB	presence/absence	binary features, short texts
ComplementNB	complement-class statistics	imbalanced text

Variant	Likelihood $P(x_j|c)$	Data
CategoricalNB	category frequencies	categorical features

Naive assumption: features independent given the class — almost never true, yet rankings are often good. Extremely fast (one pass of counting), works with tiny data, handles thousands of features. Probabilities are usually over-confident (poorly calibrated).

When to use	Scenarios
text classification baseline; very small training sets; real-time / streaming updates (partial_fit); many features	spam filtering, sentiment, news/topic categorisation, document triage, quick baselines before heavier models

7.5 Trees, Random Forest, SVM — recap

Model	One-line summary	Use when
Decision Tree (Ch. 4)	greedy if-else splits minimising impurity	need human-readable rules
Random Forest (Ch. 5)	bagged decorrelated trees, vote/average	robust tabular default
SVM (Ch. 6)	maximum-margin boundary, kernels for non-linearity	small/medium, high-dim, clear margins
Gradient Boosting (Ch. 11)	sequential trees fitting residuals	best accuracy on tabular data

7.6 Unsupervised algorithms

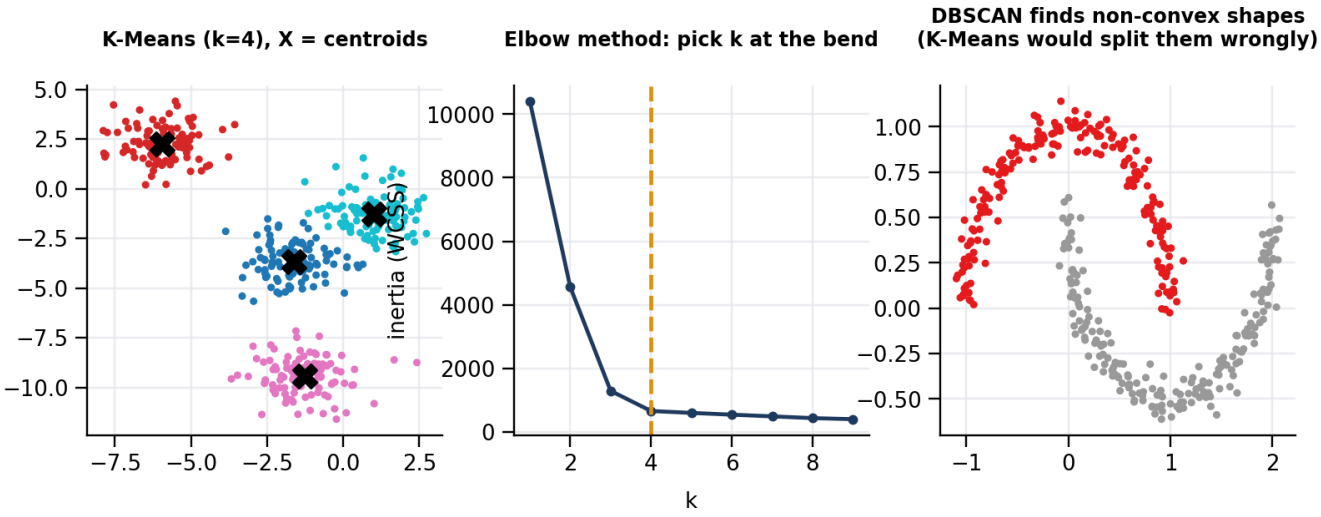


Figure: K-Means, elbow method, DBSCAN.

Algorithm	How it works	Key params	Use when / scenarios
K-Means	assign points to nearest centroid → move centroids to the mean → repeat; minimises within-cluster sum of squares $\Sigma\|x - \mu_c\|^2$	k (elbow, silhouette), init='k-means++', n_init	spherical, similar-size clusters: customer segmentation, grouping query embeddings, colour quantisation
DBSCAN	dense regions (≥ min_samples within eps) form clusters; sparse points = noise	eps, min_samples	arbitrary shapes + outliers: geo-spatial hotspots, anomaly-aware grouping
HDBSCAN	hierarchical DBSCAN over varying densities	min_cluster_size	embedding clustering (BERTopic), varying densities

Algorithm	How it works	Key params	Use when / scenarios
Hierarchical (agglomerative)	merge closest clusters bottom-up (Ward/complete/average linkage); dendrogram	n_clusters or distance threshold, linkage	small data, want a taxonomy/dendrogram
Gaussian Mixture (GMM)	soft clustering with EM; elliptical clusters; probabilities	n_components, covariance_type	overlapping clusters, density estimation
PCA	project onto directions of max variance (eigenvectors of covariance)	n_components	compression, de-correlation, visualisation (Ch. 10)
Isolation Forest	anomalies are isolated by fewer random splits	contamination	fraud, intrusion, sensor anomalies

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \in [-1, 1]$$

silhouette: a = mean distance to own cluster, b = mean distance to nearest other cluster; higher is better

7.7 Master comparison table

Algorithm	Type	Scaling needed	Interpretability	Train speed	Predict speed	Non-linear	Handles big data	Probabilities
Linear / Ridge / Lasso	reg	yes (reg.)	high	very fast	very fast	no	yes	—
Logistic Regression	clf	yes	high	very fast	very fast	no	yes	good
KNN	both	yes	medium	none	slow	yes	no (w/o ANN)	rough
Naive Bayes	clf	no	medium	very fast	very fast	limited	yes	over-confident
Decision Tree	both	no	very high	fast	very fast	yes	yes	rough
Random Forest	both	no	medium	medium	medium	yes	yes	ok
Gradient Boosting (XGB/LGBM)	both	no	medium (SHAP)	medium	fast	yes	yes	ok (calibrate)
SVM (kernel)	both	yes	low	slow	medium	yes	no	Platt
Linear SVM	clf	yes	high	fast	very fast	no	yes	Platt
K-Means	cluster	yes	high	fast	fast	no (spherical)	yes (MiniBatch)	—

7.8 Scenario → algorithm guide

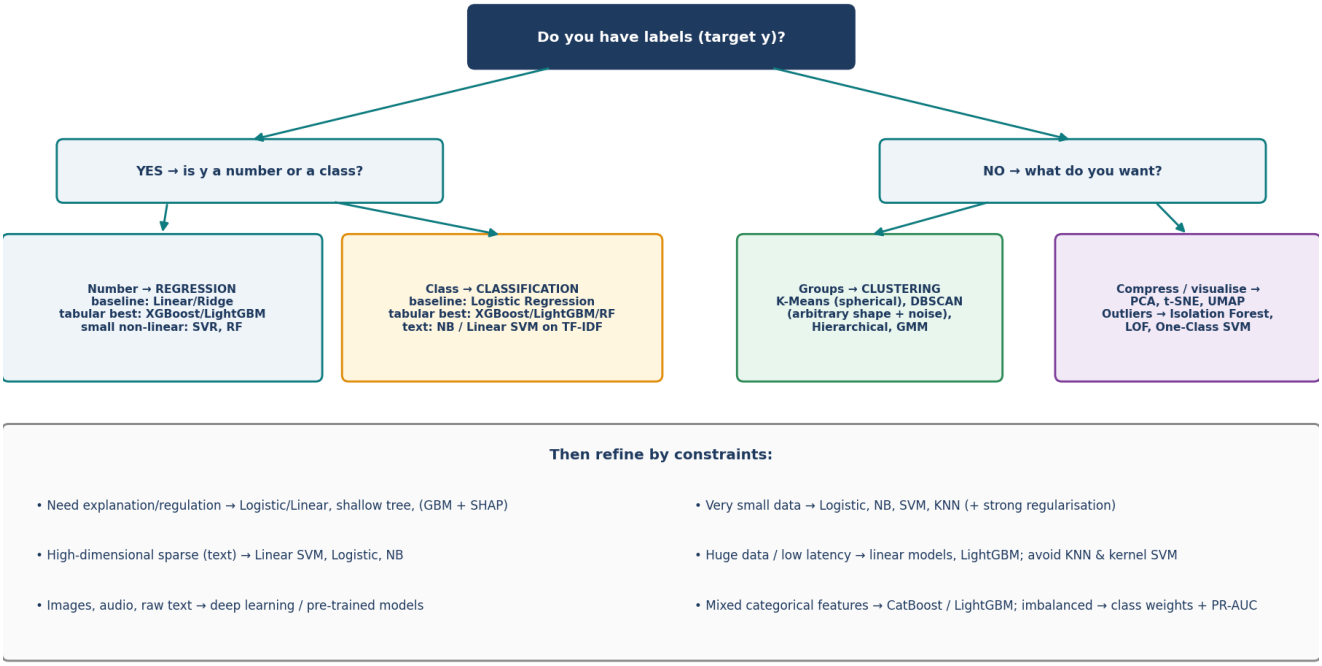


Figure: A practical decision flow for choosing an algorithm.

Scenario	Recommended	Why
Bank: rank 4 lakh customers, call top 1 lakh most likely to buy	XGBoost / LightGBM classifier → rank by probability; Logistic Regression as explainable baseline	best tabular accuracy; evaluate with lift / precision@top-25%
Credit approval that must be explained to regulators	Logistic Regression scorecard or shallow tree; GBM + SHAP if allowed	transparent coefficients / rules
Real-time fraud detection (<50 ms)	LightGBM/XGBoost (+ Isolation Forest for new patterns)	fast inference, handles imbalance with weights
Spam / sentiment / ticket text with 5k labelled examples	TF-IDF + Linear SVM / Logistic Regression / NB; or LLM embeddings + LogReg	linear models excel on sparse high-dim text
Predict house price from 20 features	Gradient boosting; Ridge baseline	non-linear interactions
300 rows of lab measurements, classify disease	Logistic Regression, SVM-RBF, small RF; strong CV	small data → simpler models, avoid overfitting
Customer segmentation without labels	K-Means on scaled RFM features (or GMM); PCA for plots	simple, interpretable segments
Group 50k user questions to find FAQ gaps	sentence embeddings → UMAP → HDBSCAN / K-Means	semantic clusters
Detect unusual server metrics	Isolation Forest, LOF, z-score rules	no labels for anomalies
Recommend similar products	KNN / ANN on embeddings, matrix factorisation	similarity-based
Image / audio / free text understanding	pre-trained deep models / LLMs	classical models need hand-made features
Route chatbot queries to 12 intents cheaply	embedding + Logistic Regression (or fine-tuned small BERT)	1000× cheaper than an LLM call per query

CHAPTER 8

Hands-on: Multi-Model Comparison & Tuning

Fair comparison protocol • baseline • comparing 8 models with the same CV • statistical comparison • Grid vs Random vs Bayesian search (Optuna) • tuning ranges per algorithm • nested CV • final model selection

8.1 Protocol for a fair comparison

1. Same train/test split, same CV folds (fixed random_state), same metric for all models.
2. Every model inside a Pipeline with its required preprocessing (scaling for SVM/KNN/LogReg, none needed for trees).
3. Start with a **dummy baseline** (DummyClassifier) and a simple model.
4. Compare mean **and** standard deviation across folds; also training time and inference time.
5. Tune only the top 2–3 candidates; evaluate the final choice once on the test set.

8.2 Code: compare many models

compare_models.py

```
import time, pandas as pd, numpy as np
from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from xgboost import XGBClassifier

models = {
    "Dummy":         DummyClassifier(strategy="most_frequent"),
    "LogReg":        make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)),
    "KNN":           make_pipeline(StandardScaler(), KNeighborsClassifier(15)),
    "NaiveBayes":    GaussianNB(),
    "DecisionTree":  DecisionTreeClassifier(max_depth=6, random_state=0),
    "RandomForest":  RandomForestClassifier(400, n_jobs=-1, random_state=0),
    "GradBoost":     GradientBoostingClassifier(random_state=0),
    "SVM-RBF":       make_pipeline(StandardScaler(), SVC(probability=True, random_state=0)),
    "XGBoost":       XGBClassifier(n_estimators=400, learning_rate=0.05, max_depth=4, eval_metric="logloss"),
}
cv = StratifiedKFold(5, shuffle=True, random_state=42); rows = []
for name, m in models.items():
    full = make_pipeline(pre, m) if name not in () else m            # 'pre' = ColumnTransformer from Chapter 2
    r = cross_validate(full, X_tr, y_tr, cv=cv, scoring=["accuracy", "f1", "roc_auc"], n_jobs=-1)
    rows.append({"model": name, **{k[5:]: f"{v.mean():.3f} ± {v.std():.3f}" for k, v in r.items() if k.startswith("test_")},
                 "fit_s": round(r["fit_time"].mean(), 2)})
print(pd.DataFrame(rows).sort_values("roc_auc", ascending=False).to_string(index=False))
```

Is the difference real? (statistical comparison)

Two models' CV means differing by 0.003 may be noise. Use the fold-wise scores: a paired t-test on the same folds (corrected resampled t-test is better), or repeated CV and check whether intervals overlap. Prefer the simpler/faster model when scores are statistically tied.

8.3 Hyper-parameter tuning methods

Method	How	Pros / cons
Manual	intuition + validation curves	learning tool; not systematic
GridSearchCV	every combination of a grid	exhaustive; explodes combinatorially
RandomizedSearchCV	sample n_iter combinations from distributions	much more efficient for many parameters (only a few matter)
Bayesian optimisation (Optuna, Hyperopt)	a surrogate model proposes promising settings using past results (TPE)	fewest trials; pruning of bad trials
Successive halving / Hyperband	evaluate many configs cheaply, keep the best, give them more resources	HalvingRandomSearchCV; fast

tuning.py

```
from sklearn.model_selection import RandomizedSearchCV, GridSearchCV
from scipy.stats import randint, uniform, loguniform
rf_space = {"n_estimators": randint(200, 1000), "max_depth": [None, 6, 10, 16], "min_samples_leaf": randint(1, 20),
            "max_features": ["sqrt", "log2", 0.3, 0.5]}
rs = RandomizedSearchCV(RandomForestClassifier(n_jobs=-1, random_state=0), rf_space, n_iter=40, cv=cv, scoring="roc_auc",
        random_state=0, n_jobs=-1)
rs.fit(Xt_tr, y_tr); print(rs.best_params_, rs.best_score_)
pd.DataFrame(rs.cv_results_).sort_values("rank_test_score")[["params", "mean_test_score", "std_test_score"]].head()

import optuna
def objective(trial):
    params = {"n_estimators": trial.suggest_int("n_estimators", 200, 1500), "learning_rate":
        trial.suggest_float("learning_rate", 0.01, 0.3, log=True),
              "max_depth": trial.suggest_int("max_depth", 3, 10), "subsample": trial.suggest_float("subsample", 0.5, 1.0),
              "colsample_bytree": trial.suggest_float("colsample_bytree", 0.5, 1.0), "reg_lambda":
        trial.suggest_float("reg_lambda", 1e-3, 10, log=True)}
    return cross_val_score(XGBClassifier(**params, eval_metric="logloss"), Xt_tr, y_tr, cv=cv, scoring="roc_auc").mean()
study = optuna.create_study(direction="maximize"); study.optimize(objective, n_trials=60); study.best_params
```

8.4 Tuning cheat sheet per algorithm

Algorithm	Most important hyper-parameters (typical search range)
Logistic Regression	C ∈ logspace(−3, 3); penalty l1/l2/elasticnet; class_weight
KNN	n_neighbors 1–50 (odd); weights uniform/distance; metric / p
Decision Tree	max_depth 2–15; min_samples_leaf 1–50; criterion; ccp_alpha
Random Forest	n_estimators 200–1000; max_features sqrt/0.3–0.7; min_samples_leaf 1–20; max_depth
SVM-RBF	C ∈ logspace(−2, 3); gamma ∈ logspace(−4, 1) or 'scale' (tune jointly)
Gradient Boosting / XGBoost	learning_rate 0.01–0.3 ↔ n_estimators (with early stopping); max_depth 3–10; subsample, colsample 0.5–1; min_child_weight; reg_lambda/alpha
LightGBM	num_leaves 15–255; min_child_samples; learning_rate; feature_fraction; bagging_fraction
Naive Bayes	alpha (smoothing) 0.01–1; var_smoothing (Gaussian)
K-Means	k; init; n_init

8.5 Nested cross-validation and final selection

nested_cv.py

```
from sklearn.model_selection import cross_val_score
inner = StratifiedKFold(3, shuffle=True, random_state=1); outer = StratifiedKFold(5, shuffle=True, random_state=2)
tuned_svm = GridSearchCV(make_pipeline(StandardScaler(), SVC()), {"svc__C": [0.1, 1, 10], "svc__gamma": ["scale", 0.01,
        0.1]}, cv=inner)
print(cross_val_score(tuned_svm, Xt_tr, y_tr, cv=outer, scoring="roc_auc").mean())   # unbiased estimate of "tune + train"
        procedure
```

Final choice = best validated score **subject to** constraints (latency, interpretability, memory, maintenance). Refit on all training data with the chosen hyper-parameters, evaluate **once** on the untouched test set, then save the pipeline.

CHAPTER 9

Bias–Variance, Overfitting & Practical Optimisation

Bias–variance decomposition • under/overfitting diagnosis • learning & validation curves • regularisation toolbox • early stopping • optimisation (gradient descent variants) • practice assessment

9.1 The bias–variance decomposition

$$\mathbb{E}\big[(y-\hat{f}(x))^2\big] = \big(\mathbb{E}[\hat{f}(x)]-f(x)\big)^2 + \mathbb{E}\big[(\hat{f}(x)-\mathbb{E}[\hat{f}(x)])^2\big] + \sigma^2$$

= Bias² + Variance + irreducible noise

- **Bias**: error from wrong assumptions — the model is too simple to represent the truth (systematic error). Example: a straight line for a curved relationship.
- **Variance**: error from sensitivity to the particular training sample — a different sample gives a very different model. Example: an unpruned tree.
- **Noise**: randomness no model can remove (label errors, missing information).

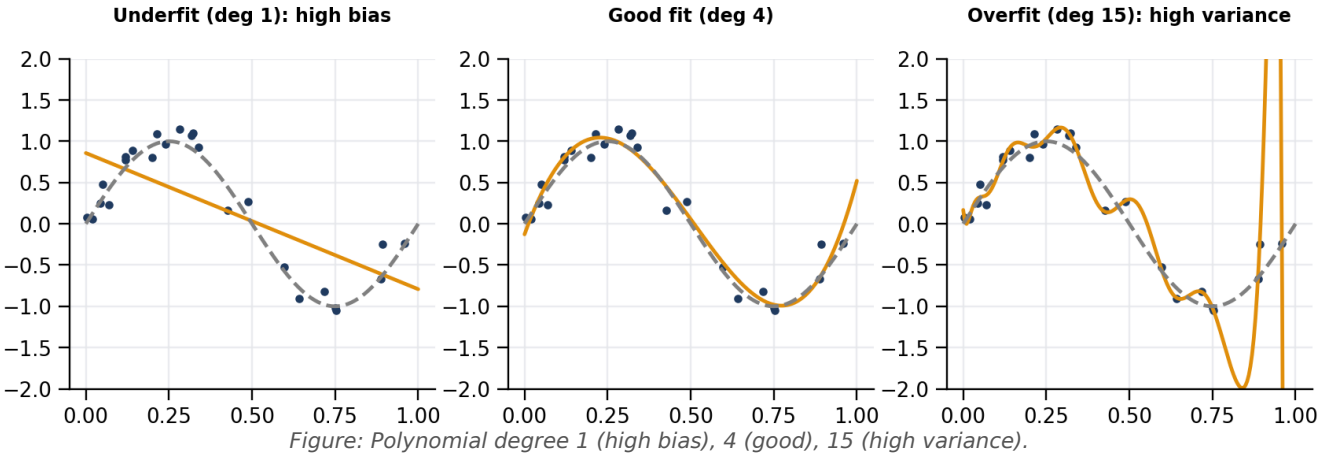


Figure: Polynomial degree 1 (high bias), 4 (good), 15 (high variance).

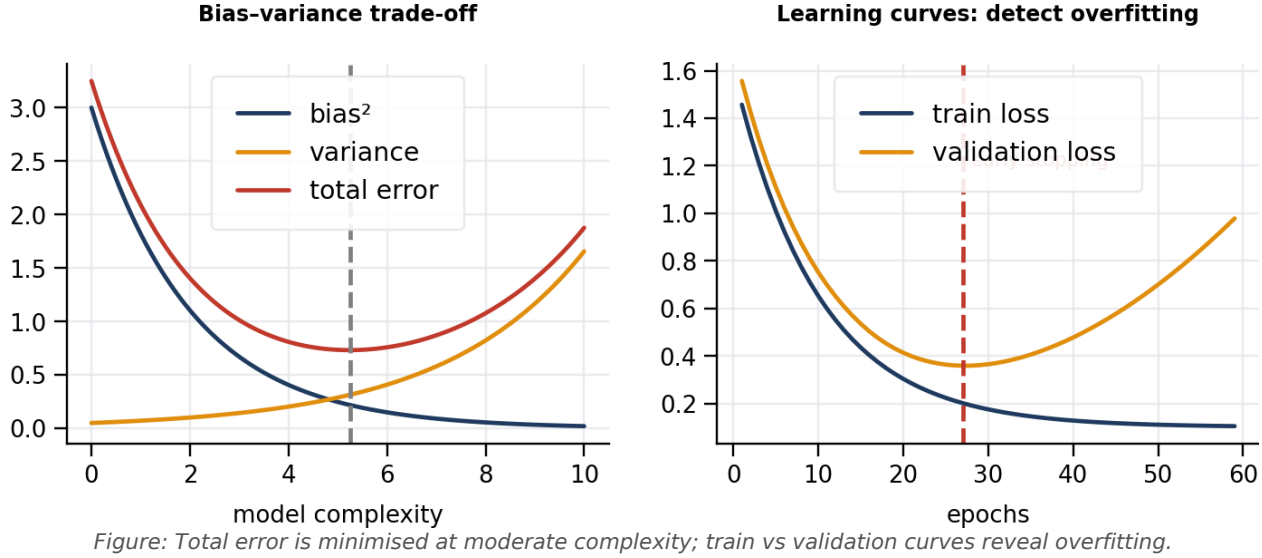


Figure: Total error is minimised at moderate complexity; train vs validation curves reveal overfitting.

9.2 Diagnosing your model

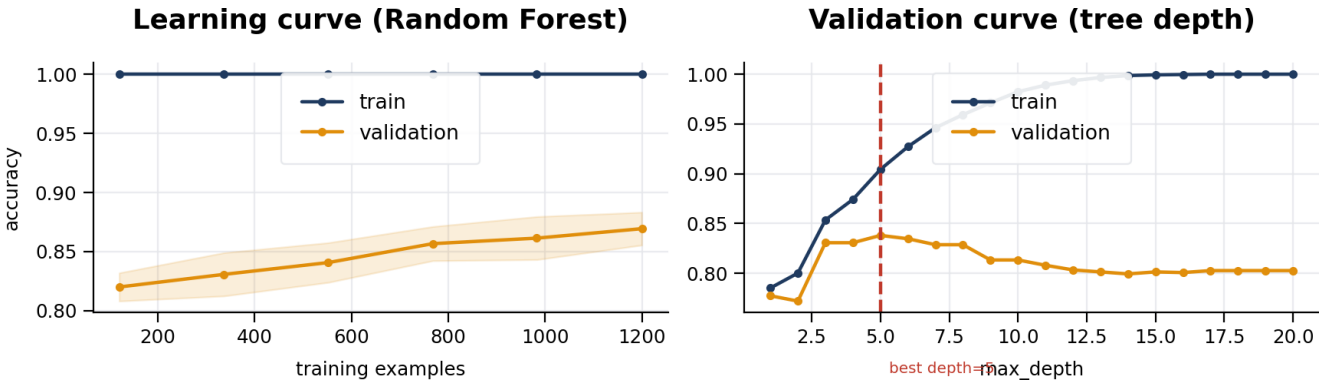


Figure: Real learning and validation curves (computed with sklearn).

Symptom	Diagnosis	What to do
train score low, val score low, close together	high bias (underfitting)	more/better features, more complex model, less regularisation (↑C, ↓alpha, ↑depth), train longer, boosting
train score high, val score much lower	high variance (overfitting)	more data, augmentation, regularisation (↓C, ↑alpha, ↓depth, ↑min_samples_leaf), feature selection, bagging/RF, early stopping, dropout (NN)
both high but test much lower	leakage or distribution shift	check pipeline, splits, time ordering, duplicates
val score jumps around between folds	high variance of estimate / small data	repeated CV, more data, simpler model
learning curve: val still rising at max data	more data will help	collect / label more
learning curve: both flat and close	more data won't help	need better features or model

9.3 Complexity knobs per algorithm

Algorithm	↑ complexity (lower bias, higher variance)	↓ complexity (higher bias, lower variance)
Linear/Logistic	more features, polynomial terms, ↑C / ↓alpha	regularise, fewer features
KNN	↓ k	↑ k
Decision tree	↑ max_depth, ↓ min_samples_leaf	prune, ↑ min_samples_leaf
Random Forest	↑ max_features, deeper trees	↓ max_features, ↑ leaf size
SVM	↑ C, ↑ γ	↓ C, ↓ γ
Boosting	↑ n_estimators, ↑ depth, ↑ learning rate	↓ learning rate + early stopping, subsampling, ↑ regularisation
Neural nets	more layers/units, more epochs	dropout, weight decay, early stopping

9.4 Practical optimisation: how models are trained

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta J(\theta_t)$$

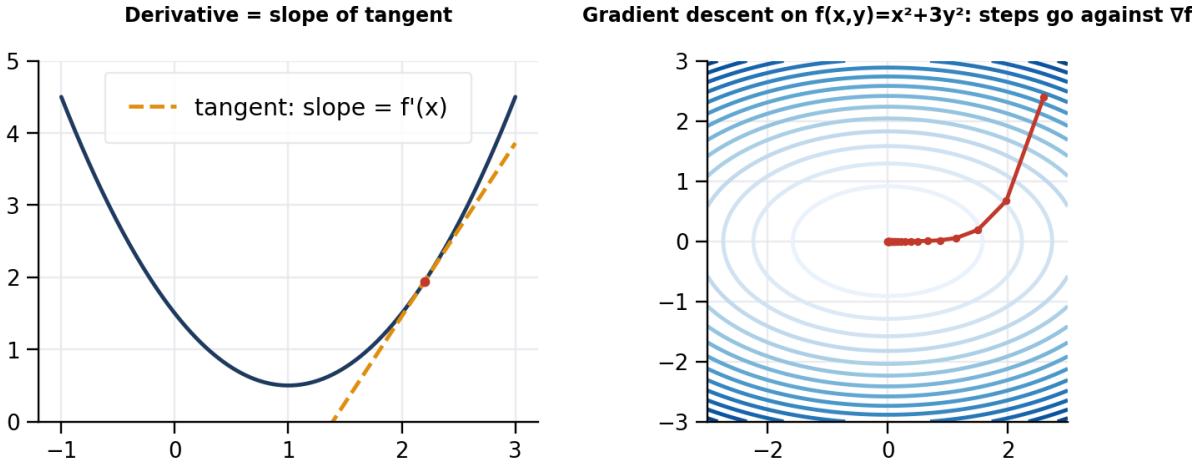


Figure: Gradient descent follows the negative gradient downhill.

Variant	Gradient computed on	Notes
Batch GD	all n samples	stable, slow per step
Stochastic GD (SGD)	one sample	noisy, fast, online learning (SGDClassifier/Regressor)
Mini-batch GD	batch of 32–1024	standard in deep learning
Momentum / Adam	adaptive steps	faster convergence (DL)
Coordinate descent	one parameter at a time	Lasso / Elastic Net
Newton / L-BFGS	uses curvature (second order)	LogisticRegression lbfgs solver

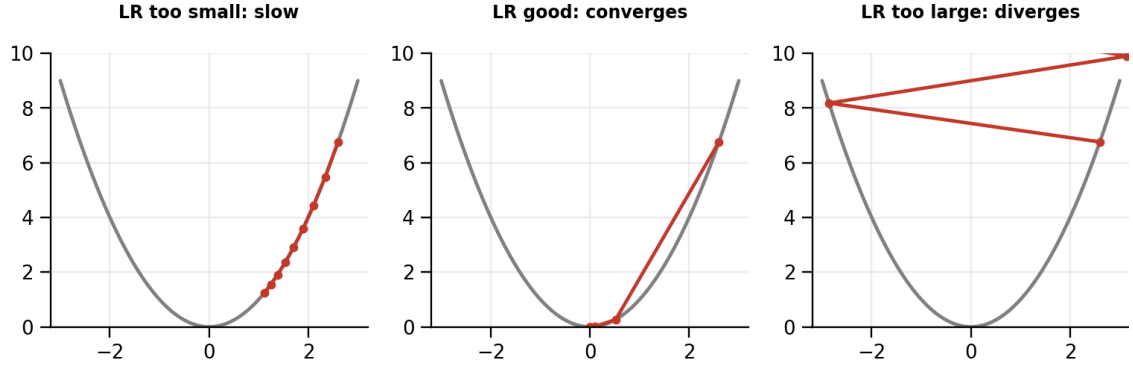


Figure: Learning rate too small, good, too large.

- Scale features so the loss surface is round, not a long valley → faster, more stable gradient descent.
- Watch convergence warnings (increase max_iter, scale data).
- Use **early stopping** for iterative learners: XGBoost early_stopping_rounds, GradientBoosting n_iter_no_change, SGD early_stopping=True.

9.5 Practice assessment (milestone-style)

Q1. A decision tree has 100% train accuracy and 72% test accuracy. Most likely issue?

Overfitting (high variance) → limit max_depth, increase min_samples_leaf, prune, or use Random Forest.

Q2. Increasing k in KNN generally…

Increases bias and decreases variance (smoother boundary).

Q3. Which reduces variance most directly: bagging or boosting?

Bagging (averaging independent models). Boosting mainly reduces bias.

Q4. In an SVM, increasing C will…

Narrow the margin and penalise violations more → lower bias, higher variance.

Q5. Lasso vs Ridge: which can set coefficients exactly to zero?

Lasso (L1).

Q6. Training and validation errors are both high and close. Next step?

Reduce bias: add features, use a more complex model, reduce regularisation.

Q7. Why fit StandardScaler inside the Pipeline?

To avoid leakage — scaler statistics come only from training folds.

Q8. RBF SVM with very large γ produces…

Highly irregular, overfit boundaries (islands around points).

Q9. For 1% fraud data, which metric?

PR-AUC / recall at fixed precision / F1 — not accuracy.

Q10. Random Forest: does adding more trees cause overfitting?

No — error plateaus; more trees only cost time. Overfitting is controlled by tree depth/leaf size.

Q11. Which models need feature scaling?

KNN, SVM, logistic/linear regression with regularisation, K-Means, PCA, neural nets. Trees and tree ensembles don't.

Q12. What is the OOB score?

Accuracy measured on rows not included in each tree's bootstrap sample — a built-in validation estimate for bagging/Random Forest.

Q13. Gini of a node with 8 positives and 2 negatives?

$1 - (0.8^2 + 0.2^2) = 0.32$.

Q14. Entropy of a 50/50 binary node?

1 bit (maximum).

Q15. Learning rate too high in gradient descent causes…

Oscillation or divergence of the loss.

Q16. Which is a lazy learner?

KNN.

Q17. Which classifier assumes conditional independence of features?

Naive Bayes.

Q18. You get 99% validation accuracy suspiciously fast. First check?

Data leakage (target-derived feature, duplicates across splits, preprocessing fit on all data).

CHAPTER 10

Feature Engineering & Dimensionality Reduction

Numeric transforms • scaling • encoding categoricals • dates • text • interactions • binning • missing values • outliers • imbalanced data • feature selection (filter, wrapper, embedded) • PCA (math + steps) • LDA • t-SNE • UMAP • when to use which

"Better data beats fancier algorithms." Feature engineering turns raw data into inputs that make patterns easy for a model to learn. For classical ML (especially linear models, KNN, SVM) it is often the biggest lever.

10.1 Numeric features

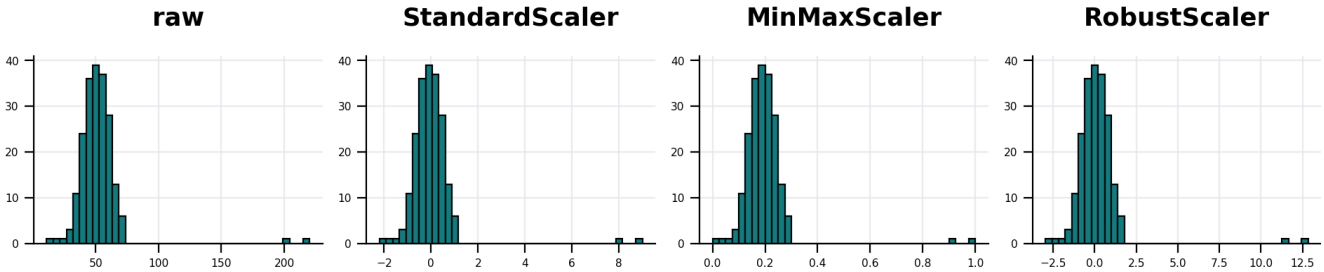


Figure: Same data with two outliers under different scalers (computed).

Technique	Formula / how	When
StandardScaler	$z = (x - \mu)/\sigma$	default for linear models, SVM, KNN, PCA, NN
MinMaxScaler	$(x - \min)/(\max - \min) \rightarrow [0,1]$	bounded inputs, NN, images; sensitive to outliers
RobustScaler	$(x - \text{median})/\text{IQR}$	data with outliers
Log / Box-Cox / Yeo-Johnson	log1p(x); PowerTransformer	right-skewed (income, counts, prices)
Binning / discretisation	KBinsDiscretizer, pd.cut / qcut	non-linear effects for linear models, robustness
Polynomial & interactions	x^2, $x_1 \cdot x_2$ (PolynomialFeatures)	linear models need explicit interactions
Ratios & domain features	debt/income, price per sq ft, clicks/impressions	often the biggest wins
Clipping / winsorising	cap at 1st/99th percentile	extreme outliers

10.2 Categorical features

Encoding	How	Pros / cons / when
One-hot	binary column per category	nominal, low cardinality; explodes for high cardinality; handle_unknown='ignore'
Ordinal / label	integer codes in order	ordered categories (low/med/high); trees can use arbitrary codes
Target (mean) encoding	replace category by mean target (with CV + smoothing)	high cardinality (pincode, merchant); leakage risk → sklearn TargetEncoder uses cross-fitting
Frequency / count	category → its frequency	high cardinality, simple
Binary / hashing	hash categories into k columns	very high cardinality, streaming
Embeddings	learned dense vectors	deep learning, entity embeddings
Native handling	CatBoost, LightGBM categorical_feature	no manual encoding needed

10.3 Date-time, text and other features

- **Dates**: year, month, day, weekday, hour, is_weekend, is_holiday, days since signup, time to event; cyclical encoding $\sin(2\pi \cdot \text{hour}/24)$, $\cos(2\pi \cdot \text{hour}/24)$.
- **Aggregations**: per-customer mean/max/count over last 7/30/90 days (RFM: recency, frequency, monetary).
- **Text**: length, word count, TF-IDF, sentiment score, sentence embeddings.
- **Geo**: distance to store, clusters of lat/long.
- **Lags & rolling stats** for time series (shift, rolling mean) — compute only from the past.

10.4 Missing values, outliers and imbalance

Problem	Options
Missing values	drop (few rows), mean/median/mode, constant 'Unknown', KNNImputer, IterativeImputer (MICE), add missing-indicator, use models that handle NaN (XGBoost, LightGBM, HistGradientBoosting)
Outliers	detect: IQR rule (< Q1 − 1.5·IQR or > Q3 + 1.5·IQR), z-score > 3, Isolation Forest; treat: verify, remove errors, cap, log-transform, robust models/losses
Imbalanced classes	stratified split; class_weight / scale_pos_weight; resampling (RandomUnderSampler, SMOTE — only on training folds via imblearn Pipeline); threshold tuning; metrics PR-AUC/F1

10.5 Feature selection

Family	Methods	Notes
Filter (model-agnostic)	variance threshold, correlation with target, chi^2 (non-negative features), ANOVA F-test, mutual information	fast; ignores feature interactions
Wrapper	Recursive Feature Elimination (RFE / RFECV), forward/backward sequential selection	uses a model; expensive; can overfit
Embedded	Lasso (L1) coefficients, tree importances (SelectFromModel)	selection during training

Family	Methods	Notes
Model-explanation based	permutation importance, SHAP	robust for final pruning

feature_selection.py

```
from sklearn.feature_selection import SelectKBest, mutual_info_classif, RFECV, SelectFromModel, VarianceThreshold
sel = SelectKBest(mutual_info_classif, k=20)                                    # filter
rfe = RFECV(LogisticRegression(max_iter=2000), step=1, cv=5, scoring="f1")     # wrapper
emb = SelectFromModel(LogisticRegression(penalty="l1", solver="liblinear", C=0.1))   # embedded (L1)
pipe = Pipeline([("pre", pre), ("sel", sel), ("clf", RandomForestClassifier())])   # selection INSIDE the
        pipeline
```

10.6 Dimensionality reduction

Curse of dimensionality: as dimensions grow, data becomes sparse, distances concentrate (nearest and farthest neighbours look alike), and models need exponentially more data. Reducing dimensions speeds training, removes noise and multicollinearity, and enables visualisation.

PCA — Principal Component Analysis

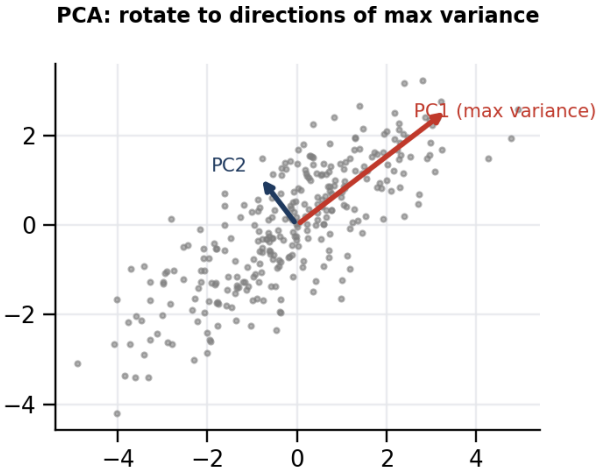


Figure: PC1 is the direction of maximum variance; PC2 is orthogonal to it.

1. Standardise X (n × d).
2. Covariance matrix C = $X^T X/(n-1)$ (d × d).
3. Eigen-decomposition C v = λ v (or SVD of X): eigenvectors = principal components, eigenvalues = variance along them.
4. Sort by eigenvalue, keep top k (explained variance ≥ 90–95% or elbow of scree plot).
5. Project: Z = X W_k.

$$\max_{\|w\|=1} \mathrm{Var}(Xw) = w^T C w \;\Rightarrow\; Cw = \lambda w, \qquad \mathrm{EVR}_k = \frac{\lambda_k}{\sum_j \lambda_j}, \qquad X = U\Sigma V^T$$

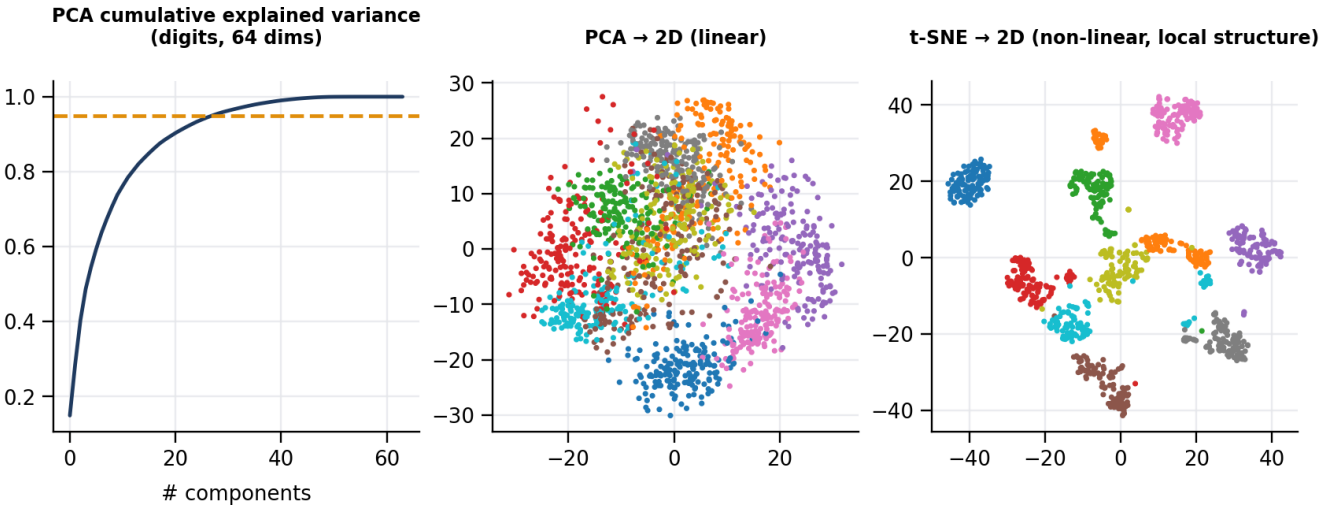


Figure: Digits (64 dims): cumulative explained variance, PCA 2-D, t-SNE 2-D (computed).

pca.py

```
from sklearn.decomposition import PCA
pca_pipe = Pipeline([("sc", StandardScaler()), ("pca", PCA(n_components=0.95)), ("clf",
        LogisticRegression(max_iter=2000))])   # keep 95% variance
pca = PCA().fit(StandardScaler().fit_transform(X)); np.cumsum(pca.explained_variance_ratio_)
pca.components_       # loadings: how each original feature contributes to each component
```

Method	Type	Use	Notes
PCA	linear, unsupervised	compression, de-noising, speed, decorrelation	components hard to interpret; scale first
Truncated SVD	linear	sparse matrices (TF-IDF → LSA)	no centring needed
LDA (Linear Discriminant Analysis)	linear, supervised	maximise class separation; ≤ (classes − 1) dims	also a classifier
Kernel PCA	non-linear	curved manifolds	costly
t-SNE	non-linear, stochastic	2-D/3-D visualisation of clusters	no transform for new data; distances between clusters not meaningful; perplexity 5–50
UMAP	non-linear	visualisation + preprocessing; faster, preserves more global structure	n_neighbors, min_dist
Autoencoders	non-linear (NN)	learned compression, anomaly detection	needs more data

Interview tip: Feature selection keeps a subset of the original (interpretable) features; feature extraction (PCA etc.) creates new combined features. Trees rarely need PCA; linear models, KNN and SVM often benefit.

CHAPTER 11

Ensemble Learning: Bagging & Boosting

Why ensembles work • voting • bagging • Random Forest • Extra Trees • boosting idea • AdaBoost (math) • Gradient Boosting (math) • XGBoost • LightGBM • CatBoost • HistGradientBoosting • stacking & blending • comparison & when to use

11.1 Why combining models works

If models make **different** errors, combining them cancels errors. With M independent classifiers each 70% accurate, a majority vote of 11 is ~92% accurate. In practice models are correlated, so the key is **diversity**: different data samples, features, algorithms or hyper-parameters.

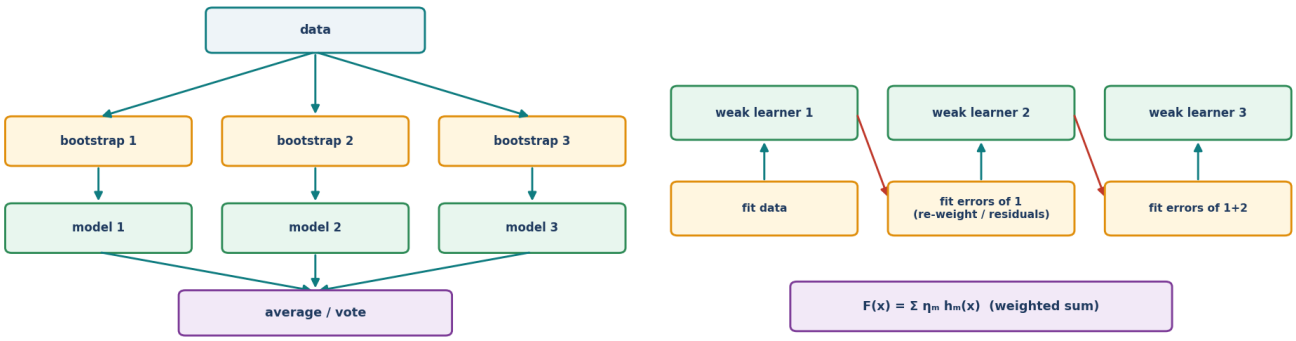


Figure: Bagging (parallel, variance reduction) vs boosting (sequential, bias reduction).

	Bagging	Boosting	Stacking
Training	parallel, independent	sequential, each depends on previous	base models in parallel + meta-model
Data	bootstrap samples	re-weighted samples / residuals	out-of-fold predictions
Base learners	strong, high-variance (deep trees)	weak, high-bias (shallow trees, stumps)	diverse strong models
Reduces	variance	bias (and variance)	both
Combination	average / vote	weighted sum	learned by meta-model
Overfitting risk	low	higher (needs early stopping, LR)	medium
Examples	Random Forest, Bagged trees, Extra Trees	AdaBoost, GBM, XGBoost, LightGBM, CatBoost	StackingClassifier

11.2 Voting

voting.py

```
from sklearn.ensemble import VotingClassifier
vote = VotingClassifier([("lr", lr_pipe), ("rf", RandomForestClassifier(300)), ("svm", svm_pipe)], voting="soft")   # soft
        = average probabilities (needs predict_proba); hard = majority label
```

11.3 Bagging

Bootstrap AGGregatING: train the same algorithm on B bootstrap samples, average predictions. Works best with unstable, low-bias models (deep trees). Random Forest = bagging + random feature subsets (Chapter 5); Extra Trees adds random thresholds.

bagging.py

```
from sklearn.ensemble import BaggingClassifier
bag = BaggingClassifier(DecisionTreeClassifier(), n_estimators=200, max_samples=0.8, max_features=0.8, oob_score=True,
        n_jobs=-1)
```

11.4 AdaBoost (Adaptive Boosting)

1. Initialise sample weights $w_i = 1/n$.
2. For m = 1 … M: fit a weak learner h_m (often a depth-1 stump) on weighted data.
3. Weighted error $\varepsilon_m = \Sigma\, w_i \cdot [h_m(x_i) \neq y_i] \,/\, \Sigma\, w_i$.
4. Learner weight $\alpha_m = \tfrac{1}{2} \ln((1 - \varepsilon_m)/\varepsilon_m)$ (accurate learners get more say).
5. Update $w_i \leftarrow w_i \cdot \exp(-\alpha_m\, y_i\, h_m(x_i))$ → misclassified points gain weight; normalise.
6. Final: $H(x) = \text{sign}(\Sigma\, \alpha_m\, h_m(x))$.

Worked step: 10 samples, stump misclassifies 3 → $\varepsilon = 0.3$ → $\alpha = \tfrac{1}{2} \ln(0.7/0.3) = 0.424$. Misclassified weights × $e^{0.424} = 1.53$; correct × $e^{-0.424} = 0.65$; after normalising, the 3 hard points carry 50% of the total weight.

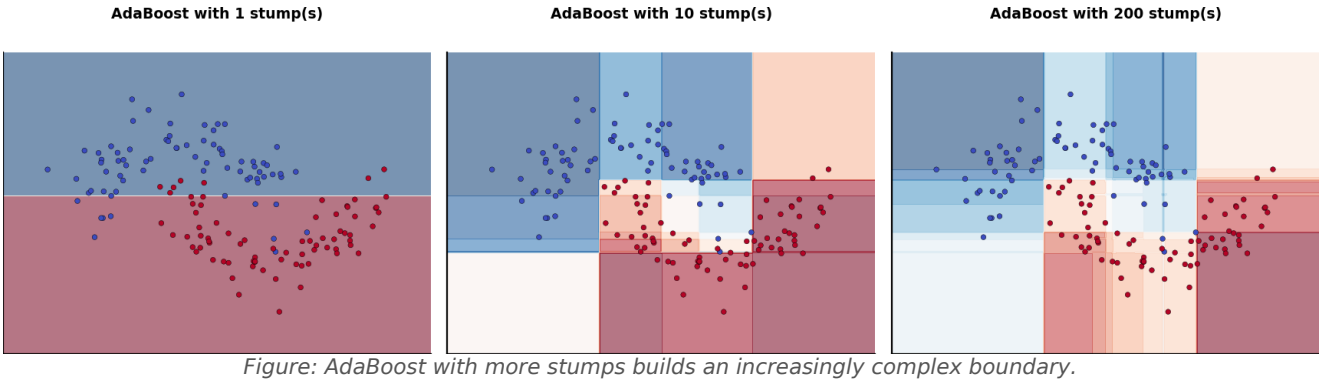


Figure: AdaBoost with more stumps builds an increasingly complex boundary.

AdaBoost minimises exponential loss; it is sensitive to noisy labels and outliers (they get huge weights).

11.5 Gradient Boosting

$$F_0(x) = \arg\min_c \sum_i L(y_i, c), \qquad r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{m-1}}, \qquad F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$$

Each new tree h_m is fit to the **negative gradient** (pseudo-residuals) of the loss — gradient descent in function space. For squared error, r = y − F(x): literally the remaining errors. For log-loss, r = y − p. The learning rate η (shrinkage, 0.01–0.3) scales each tree's contribution: smaller η needs more trees but generalises better.

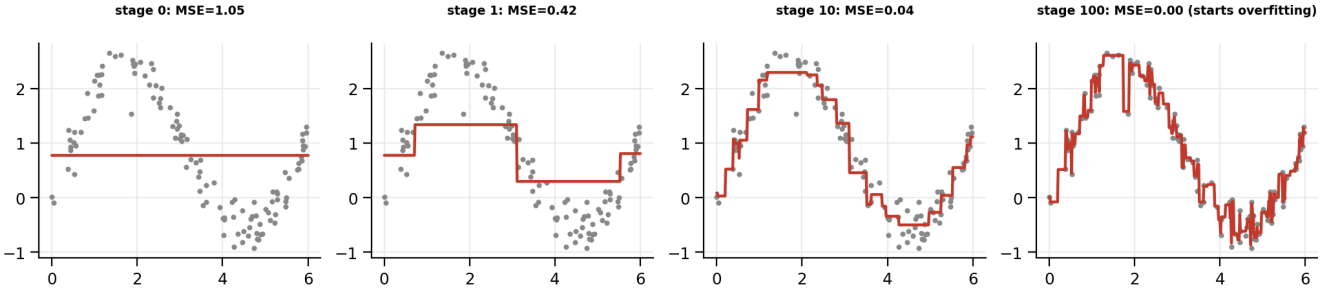


Figure: Gradient boosting with depth-2 trees and η = 0.5 (computed): the fit improves stage by stage, then starts fitting noise.

11.6 XGBoost, LightGBM, CatBoost

$$\text{Obj}^{(m)} \approx \sum_i \left[g_i f_m(x_i) + \frac{1}{2} h_i f_m^2(x_i)\right] + \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2, \qquad w_j^* = -\frac{\sum_{i \in j} g_i}{\sum_{i \in j} h_i + \lambda}$$

$$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}\right] - \gamma$$

XGBoost: second-order Taylor approximation (gradients g, Hessians h), regularised leaf weights, split gain

Library	Key innovations	Strengths
XGBoost	2nd-order optimisation, L1/L2 on leaf weights, γ min split gain, sparsity-aware (learns default direction for missing values), column/row subsampling, histogram method, parallel split finding	accurate, mature, widely used; GPU
LightGBM	histogram binning, **leaf-wise** growth (best leaf first), GOSS (keep large-gradient rows), EFB (bundle sparse features), native categoricals	very fast on large data; control num_leaves to avoid overfitting
CatBoost	ordered boosting and ordered target statistics for categoricals (no leakage), symmetric (oblivious) trees	best with many categorical features, good defaults, less tuning
HistGradientBoosting (sklearn)	LightGBM-style histograms, native NaN & categorical support	no extra dependency, fast

boosting_libs.py

```
import xgboost as xgb, lightgbm as lgb
xgbm = xgb.XGBClassifier(n_estimators=2000, learning_rate=0.03, max_depth=5, subsample=0.8, colsample_bytree=0.8,
        min_child_weight=3,
                        reg_lambda=1.0, scale_pos_weight=neg / pos, eval_metric="aucpr", early_stopping_rounds=100)
xgbm.fit(X_tr, y_tr, eval_set=[(X_val, y_val)], verbose=200)       # stops when validation stops improving
print(xgbm.best_iteration)
lgbm = lgb.LGBMClassifier(n_estimators=3000, learning_rate=0.03, num_leaves=31, min_child_samples=30, subsample=0.8,
        subsample_freq=1, colsample_bytree=0.8)
lgbm.fit(X_tr, y_tr, eval_set=[(X_val, y_val)], callbacks=[lgb.early_stopping(100)])
from catboost import CatBoostClassifier
cb = CatBoostClassifier(iterations=2000, learning_rate=0.05, depth=6, cat_features=cat_cols, early_stopping_rounds=100,
        verbose=200).fit(X_tr, y_tr, eval_set=(X_val, y_val))
```

11.7 Stacking and blending

Stacking: train diverse base models; generate their **out-of-fold** predictions on training data (to avoid leakage); train a meta-model (often Logistic Regression) on those predictions. **Blending** uses a hold-out set instead of OOF. Gains are usually small but consistent — popular in competitions, less so in production (complexity).

stacking.py

```
from sklearn.ensemble import StackingClassifier
stack = StackingClassifier(estimators=[("rf", RandomForestClassifier(400)), ("xgb", xgbm_no_es), ("svm", svm_pipe)],
                           final_estimator=LogisticRegression(), cv=5, stack_method="predict_proba", passthrough=False)
```

11.8 Which ensemble when?

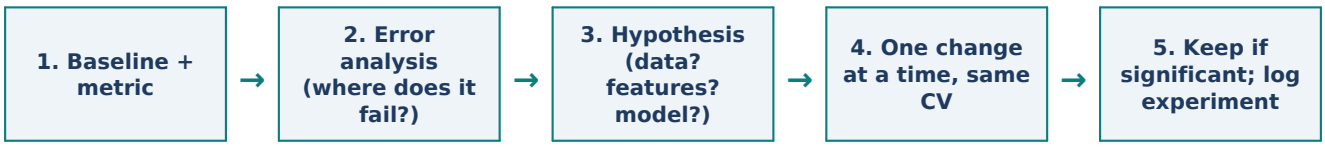
Situation	Choice
Robust default, little tuning, parallel training	Random Forest
Maximum accuracy on tabular data	XGBoost / LightGBM / CatBoost with early stopping + tuning
Many categorical features	CatBoost (or LightGBM native categoricals)
Very large dataset, speed	LightGBM / HistGradientBoosting
Noisy labels / outliers	Random Forest (boosting chases noise; AdaBoost especially)
Squeeze last 0.5% (competitions)	stacking/blending diverse models
Need interpretability	single tree / linear; or GBM + SHAP

CHAPTER 12

Practical Model Improvement Strategies

A systematic improvement loop • data-centric improvements • feature improvements • model improvements • threshold & calibration • error analysis • interpretability with SHAP • reproducibility • checklist

12.1 The improvement loop



Prioritise by expected gain ÷ effort. Typical order of impact on tabular problems: **fix data issues & leakage → better features → right model family (boosting) → tuning → ensembling → threshold/calibration**.

12.2 Data-centric strategies

- Fix label errors (review high-loss / confidently wrong samples; cleanlab).
- Collect more data for the failure segments found in error analysis; active learning (label the most uncertain samples).
- Remove duplicates and leakage; ensure train/test come from the same distribution as production.
- Handle imbalance properly (weights, resampling inside CV, thresholds).
- Augment data where meaningful (text paraphrases, image augmentation, SMOTE for tabular — cautiously).

12.3 Feature-centric strategies

- Domain features (ratios, aggregates, time since, counts); interactions for linear models.
- Better encodings for high-cardinality categoricals (target encoding with cross-fitting, CatBoost).
- Transform skewed features (log), scale for distance-based models.
- Remove noisy/irrelevant features (permutation importance < 0, RFECV).
- Use embeddings (text, entities) as features.

12.4 Model-centric strategies

- Try the right family: GBMs for tabular, linear for sparse text, SVM/LogReg for small data.
- Tune the important hyper-parameters (Chapter 8) with Random/Bayesian search.
- Regularise based on the bias–variance diagnosis (Chapter 9).
- Ensembles: Random Forest/boosting; average a few diverse strong models.
- Early stopping for boosting; calibrate probabilities.

12.5 Decision-level strategies

- Tune the classification threshold to the business cost (Chapter 3).
- Cost-sensitive learning (class_weight, sample_weight).
- Reject option: send low-confidence predictions to humans.
- Segment-specific models or thresholds when behaviour differs (new vs existing customers).

12.6 Error analysis & interpretability

shap_error_analysis.py

```
import shap
explainer = shap.TreeExplainer(xgbm)
sv = explainer.shap_values(X_val)
shap.summary_plot(sv, X_val)                      # global: which features matter and in which direction
shap.waterfall_plot(explainer(X_val)[0])          # local: why THIS customer got this score

err = X_val.assign(y=y_val, p=xgbm.predict_proba(X_val)[:, 1]); err["wrong"] = (err.p > thr) != err.y
err.groupby("Contract")["wrong"].mean()           # which segment fails most?
```

12.7 Reproducibility & experiment tracking

- Fix random seeds; version data and code; pin library versions.
- Log every experiment (params, metrics, data version, features) with MLflow or a spreadsheet.
- Keep a fixed test set; never tune on it.
- Write the model card: data, metric, limitations, fairness checks.

12.8 Improvement checklist

✔	Check
☐	Is the metric aligned with the business cost?
☐	Beats a dummy and a simple baseline?
☐	No leakage (pipeline, time order, duplicates, target-derived features)?
☐	Learning/validation curves analysed (bias or variance)?
☐	Error analysis by segment done?
☐	Feature importance sane (no ID-like features on top)?
☐	Several model families compared on identical CV?
☐	Hyper-parameters tuned on validation only; test used once?
☐	Threshold and calibration set for deployment?
☐	Pipeline saved with versions; API tested; monitoring planned?

CHAPTER 13

Interview Questions & Answers

Fundamentals • pipeline & evaluation • trees • Random Forest • SVM • other algorithms • bias–variance • features • ensembles • scenarios

13.1 Fundamentals, pipeline & evaluation

Q1. Parameters vs hyper-parameters?

Parameters are learned from data (weights, split thresholds); hyper-parameters are set before training (C, max_depth, k) and tuned on validation data.

Q2. Why separate validation and test sets?

Validation is used repeatedly to choose models and hyper-parameters, so it becomes optimistically biased; the untouched test set gives the final unbiased estimate.

Q3. What is data leakage? Give examples.

Information unavailable at prediction time leaking into training: scaling on full data, target-derived features, future data in time series, duplicates across splits. It inflates offline scores.

Q4. Why use sklearn Pipeline?

It chains preprocessing and model so transformers are fit only on training folds (no leakage), enables joint tuning, and produces one deployable object.

Q5. Precision vs recall, with examples.

Precision = correctness of positive predictions (important for spam-to-inbox or limited call budgets); recall = coverage of actual positives (important for fraud, disease).

Q6. When is ROC-AUC misleading?

With heavy class imbalance it can look high while precision is poor; PR-AUC focuses on the positive class.

Q7. How do you choose the decision threshold?

On validation data, maximise F1 or minimise expected cost ($c_{FP} \cdot FP + c_{FN} \cdot FN$), or meet a required precision/recall.

Q8. Pickle vs joblib vs ONNX?

Pickle serialises any Python object; joblib is more efficient for numpy-heavy sklearn models; ONNX is a portable, framework-independent format for fast inference. Never load untrusted pickles.

Q9. How do you serve an ML model?

Load the saved pipeline once at startup in a FastAPI app, validate inputs with Pydantic, expose /predict and /health, containerise with Docker, log predictions and monitor drift.

13.2 Decision trees & Random Forest

Q10. How does a decision tree choose a split?

For every feature and candidate threshold it computes the weighted impurity of the children and picks the split with the largest impurity decrease (information gain); it is greedy.

Q11. Gini vs entropy?

Both measure impurity and give similar trees; Gini is cheaper (no log), entropy comes from information theory.

Q12. Why do trees overfit and how do you stop it?

They can keep splitting until leaves are pure, memorising noise. Limit max_depth, min_samples_leaf, max_leaf_nodes, or prune with ccp_alpha; or use ensembles.

Q13. Do trees need feature scaling?

No — splits compare a feature to a threshold, so monotonic transformations don't change them.

Q14. How does a regression tree predict?

It splits to minimise variance (MSE) and predicts the mean target of the leaf — a piecewise-constant function.

Q15. What makes Random Forest better than one tree?

Averaging many trees trained on bootstrap samples with random feature subsets reduces variance while keeping bias low; decorrelation is key.

Q16. Role of max_features in Random Forest?

It controls how many features each split considers: lower means more diverse, less correlated trees (lower variance) but each tree is weaker.

Q17. What is OOB error?

Each tree skips ~36.8% of rows; predicting those rows with only the trees that didn't see them gives a built-in validation estimate.

Q18. Can adding trees overfit a Random Forest?

No; the error converges. Overfitting is controlled by tree complexity, not the number of trees.

Q19. Impurity vs permutation importance?

Impurity importance sums split gains on training data and is biased toward high-cardinality features; permutation importance measures the drop in validation score when a feature is shuffled and is more reliable.

13.3 SVM

Q20. What is the intuition behind SVM?

Find the hyperplane that separates classes with the maximum margin; only the support vectors define it, which improves generalisation.

Q21. What are support vectors?

Training points on or inside the margin (or misclassified) — they have non-zero Lagrange multipliers and alone determine the boundary.

Q22. Explain C.

The penalty for margin violations: large C gives a narrow margin fitting training data closely (risk of overfitting); small C gives a wider, more regularised margin.

Q23. Explain the kernel trick.

Because the SVM dual depends only on dot products, we replace them with a kernel $K(x, x') = \varphi(x)^T\varphi(x')$, learning non-linear boundaries in a high-dimensional space without computing φ.

Q24. RBF γ effect?

γ sets the reach of each point's influence: small γ gives smooth boundaries (underfit), large γ gives very local, wiggly boundaries (overfit). Tune with C.

Q25. Which kernel would you start with?

Linear for high-dimensional sparse data like text; RBF for low/medium-dimensional dense data with non-linear patterns.

Q26. Why scale features for SVM?

Margins and kernels depend on distances/dot products; unscaled features with large ranges dominate.

Q27. Limitations of SVM?

Slow on large datasets, sensitive to C/γ, no native probabilities, harder to interpret.

Q28. What is hinge loss?

max(0, 1 − y·f(x)): zero for points correctly classified beyond the margin, growing linearly for violations.

13.4 Other algorithms

Q29. Why is logistic regression a classifier?

It models the log-odds linearly and passes it through a sigmoid to produce a probability, which is thresholded to a class.

Q30. Why not MSE for logistic regression?

With a sigmoid, MSE makes the loss non-convex with weak gradients; log-loss is convex and corresponds to maximum likelihood.

Q31. Interpret a logistic coefficient of 0.7.

A one-unit increase multiplies the odds of the positive class by e^0.7 ≈ 2.0, holding other features constant.

Q32. L1 vs L2?

L1 drives some coefficients exactly to zero (feature selection); L2 shrinks all coefficients smoothly and handles multicollinearity.

Q33. Assumptions of linear regression?

Linearity, independent errors, homoscedasticity, normally distributed residuals (for inference), and low multicollinearity.

Q34. How does KNN work and how do you pick k?

It predicts from the k closest training points; pick k by cross-validation — small k overfits, large k underfits; scale features first.

Q35. Why is Naive Bayes 'naive' yet effective?

It assumes features are independent given the class; the assumption is wrong but the ranking of classes is often right, and it is fast and data-efficient, especially for text.

Q36. K-Means limitations?

Must choose k, assumes spherical similar-sized clusters, sensitive to scale, outliers and initialisation; use k-means++, scaling, or DBSCAN/GMM for other shapes.

Q37. How do you choose k in K-Means?

Elbow of inertia, silhouette score, and business interpretability.

13.5 Bias–variance, features & ensembles

Q38. Explain the bias–variance trade-off.

Expected error = bias² + variance + noise. Simple models underfit (high bias); complex models overfit (high variance); we choose complexity that minimises total error.

Q39. How do you detect overfitting?

A large gap between training and validation scores, or learning curves where validation stops improving while training keeps improving.

Q40. How do you fix underfitting?

Add informative features, use a more flexible model, reduce regularisation, train longer.

Q41. Which models need scaling?

Distance or gradient based: KNN, SVM, K-Means, PCA, regularised linear/logistic regression, neural networks — not tree-based models.

Q42. One-hot vs target encoding?

One-hot for low-cardinality nominal features; target encoding for high cardinality, computed out-of-fold with smoothing to avoid leakage.

Q43. PCA steps?

Standardise, compute covariance, eigen-decompose, sort components by variance, keep top k, project.

Q44. Feature selection vs dimensionality reduction?

Selection keeps a subset of original interpretable features; reduction creates new combined features (PCA).

Q45. Bagging vs boosting?

Bagging trains models in parallel on bootstrap samples and averages them to reduce variance; boosting trains weak models sequentially, each correcting the previous errors, to reduce bias.

Q46. How does gradient boosting work?

Start with a constant prediction, then repeatedly fit a small tree to the negative gradient of the loss (residuals for MSE) and add it scaled by a learning rate.

Q47. What makes XGBoost special?

Second-order gradients, regularised objective, shrinkage, row/column subsampling, sparsity-aware missing-value handling, histogram/parallel split finding.

Q48. LightGBM vs XGBoost?

LightGBM grows trees leaf-wise with histogram binning, GOSS and EFB, making it faster on large data; it can overfit small data unless num_leaves/min_child_samples are controlled.

Q49. Why learning rate + early stopping in boosting?

A small learning rate makes each tree contribute little (better generalisation) and early stopping picks the number of trees where validation performance peaks.

Q50. What is stacking?

Training a meta-model on out-of-fold predictions of diverse base models to learn how to best combine them.

13.6 Scenario questions

Q51. A bank wants to call the 1 lakh most likely buyers out of 4 lakh customers. Which model and how?

Binary classifier (XGBoost/LightGBM, with Logistic Regression as baseline) trained on past campaign outcomes; score all customers, rank by probability, call the top 1 lakh; evaluate with lift/gain charts and precision at the top 25%.

Q52. Model performs well offline but poorly in production. Why?

Training–serving skew, data drift, leakage in offline evaluation, different preprocessing, or label definition changes.

Q53. You have 500 rows and 2,000 features.

High risk of overfitting: use strong regularisation (L1/L2 logistic, linear SVM), feature selection inside CV, repeated CV; avoid deep trees and KNN.

Q54. 1 million rows of tabular data, need best accuracy fast.

LightGBM or XGBoost (hist) with early stopping and randomised/Bayesian tuning.

Q55. Stakeholders need to understand every decision.

Logistic regression or a shallow decision tree; if using GBM, provide SHAP explanations and reason codes.

Q56. Your classifier's top feature is customer_id.

Leakage or overfitting to an identifier — drop ID-like features and re-check splits.

CHAPTER 14

Cheat Sheet

Algorithms at a glance • formulas • sklearn one-liners

14.1 Formulas

Linear regression	$\hat{y} = w^T x + b, \quad w = (X^T X)^{-1} X^T y$
Logistic regression	$p = \sigma(w^T x + b)$
Log-loss	$-[y \log p + (1-y)\log(1-p)]$
Ridge / Lasso	$+\alpha \lVert w \rVert_2^2 \ / \ + \alpha \lVert w \rVert_1$
Gini / Entropy	$1 - \sum p_k^2 \ / \ - \sum p_k \log_2 p_k$
Info gain	$I(S) - \sum \frac{\lvert S_c \rvert}{\lvert S \rvert} I(S_c)$
SVM	$\min \frac{1}{2} \lVert w \rVert^2 + C \sum \max(0, 1 - y_i f(x_i))$
RBF kernel	$\exp(-\gamma \lVert x - x' \rVert^2)$
Naive Bayes	$\arg\max_c \log P(c) + \sum \log P(x_j \mid c)$
KNN distance	$\left(\sum \lvert x_j - z_j \rvert^p \right)^{1/p}$
RF variance	$\rho \sigma^2 + \frac{1-\rho}{B} \sigma^2$
AdaBoost α	$\frac{1}{2} \ln \frac{1-\varepsilon}{\varepsilon}$
Gradient boosting	$F_m = F_{m-1} + \eta h_m, \ h_m \approx -\nabla L$
Bias-variance	$\text{Bias}^2 + \text{Var} + \sigma^2$
F1	$\frac{2PR}{P+R}$
R²	$1 - \frac{SS_{res}}{SS_{tot}}$
PCA	$Cw = \lambda w, \quad Z = XW_k$

14.2 Algorithms at a glance

Algorithm	Key idea	Main knobs	Best scenario
Linear/Ridge/Lasso	weighted sum, least squares	alpha	linear trends, baselines
Logistic Regression	sigmoid of linear score	C, penalty	explainable classification, text, embeddings
KNN	vote of nearest neighbours	k, metric	small low-dim data, similarity
Naive Bayes	Bayes + independence	alpha	text, tiny data
Decision Tree	greedy impurity splits	max_depth, min_samples_leaf	rules, interpretability
Random Forest	bagged random trees	n_estimators, max_features	robust tabular default
SVM	max margin + kernels	C, γ, kernel	small/medium high-dim data
AdaBoost	re-weight hard samples	n_estimators, LR	clean data, weak learners
XGBoost/LightGBM/CatBoost	trees fit gradients	LR, n_estimators, depth/leaves	best tabular accuracy
K-Means / DBSCAN	centroids / density	k / eps	segmentation / shapes+noise
PCA	max-variance projection	n_components	compression, visualisation

Watch out: Code in this guide is reference-quality and was not run in your environment (except the figures, which were computed with scikit-learn). Library APIs (XGBoost early stopping, LightGBM callbacks, sklearn parameter names) change between versions — check your installed versions.